

Fəsil 12. TensorFlow ilə Xüsusi Modellər və Təlim

İndiyə qədər biz yalnız TensorFlow-un yüksək səviyyəli API-si olan Keras-dan istifadə etdik, lakin bu, bizi kifayət qədər irəli apardı: biz müxtəlif neyron şəbəkə arxitekturaları qurduq, o cümlədən regression və classification şəbəkələri, Wide & Deep şəbəkələri və self-normalizing şəbəkələr, batch normalization, dropout və learning rate schedules kimi müxtəlif texnikalardan istifadə etdik. Əslində, qarşılaşacağınız istifadə hallarının 95%-i Keras-dan (və `tf.data`; bax Fəsil 13) başqa heç nə tələb etməyəcək. Lakin indi TensorFlow-a daha dərinə dalmaq və onun aşağı səviyyəli Python API-sinə nəzər salmaq vaxtıdır. Bu, xüsusi loss funksiyaları, xüsusi metrikalar, qatlar, modellər, ilkinləşdiricilər, regularizatorlar, çəki məhdudiyyətləri və daha çoxunu yazmaq üçün əlavə nəzarətə ehtiyacınız olduqda faydalı olacaq. Hətta təlim dövrünü (training loop) tamamilə idarə etmək lazım gələ bilər; məsələn, qradiyentlərə xüsusi transformasiyalar və ya məhdudiyyətlər tətbiq etmək (sadəcə clipping etməkdən başqa) və ya şəbəkənin müxtəlif hissələri üçün çoxsaylı optimizatorlardan istifadə etmək. Bu fəsildə bütün bu halları əhatə edəcəyik, həmçinin TensorFlow-un avtomatik qrafik yaratma xüsusiyyətindən istifadə edərək xüsusi modellərinizi və təlim alqoritmlərinizi necə sürətləndirə biləcəyinizə baxacağıq. Lakin əvvəlcə TensorFlow-a qısa bir tur edək.

TensorFlow-a Qısa Bir Tur

Bildiyiniz kimi, TensorFlow böyük miqyaslı maşın öyrənmə üçün xüsusilə uyğun və incə tənzimlənmiş, ədədi hesablamalar üçün güclü bir kitabxanadır (lakin onu ağır hesablamalar tələb edən hər hansı digər iş

üçün də istifadə edə bilərsiniz). Google Brain komandası tərəfindən hazırlanmışdır və Google Cloud Speech, Google Photos və Google Search kimi Google-un bir çox böyük miqyaslı xidmətlərini gücləndirir. 2015-ci ilin noyabrında açıq qaynaq (open source) edilmişdir və hazırda sənayedə ən çox istifadə olunan deep learning kitabxanasıdır: saysız-hesabsız layihələr TensorFlow-dan şəkil təsnifatı, təbii dil emalı, tövsiyə sistemləri və zaman seriyası proqnozlaşdırması kimi hər cür maşın öyrənmə tapşırıqları üçün istifadə edir.

Bəs TensorFlow nə təklif edir? Burada bir xülasə var:

- Onun əsası NumPy-ə çox bənzəyir, lakin GPU dəstəyi ilə.
- O, paylanmış hesablamaları (çoxsaylı cihazlar və serverlər arasında) dəstəkləyir.
- O, bir növ just-in-time (JIT) kompilyatoru ehtiva edir ki, bu da hesablamaları sürət və yaddaş istifadəsi üçün optimallaşdırmağa imkan verir. Bu, Python funksiyasından *computation graph* (hesablama qrafiki) çıxarmaq, onu optimallaşdırmaq (məsələn, istifadə olunmayan nodları kəsməklə) və onu səmərəli şəkildə işə salmaqla (məsələn, müstəqil əməliyyatları avtomatik paralel işə salmaqla) işləyir.
- Computation graphs portativ formata ixrac edilə bilər, beləliklə siz TensorFlow modelini bir mühitdə (məsələn, Linux-da Python istifadə edərək) öyrədə və başqa bir mühitdə (məsələn, Android cihazında Java istifadə edərək) işlədə bilərsiniz.
- O, reverse-mode autodiff (bax Fəsil 10 və Əlavə B) tətbiq edir və RMSProp və Nadam (bax Fəsil 11) kimi əla optimizatorlar təqdim edir, beləliklə siz asanlıqla hər cür loss funksiyasını minimuma endirə bilərsiniz.

TensorFlow bu əsas xüsusiyyətlər üzərində qurulmuş daha çox funksiyalar təklif edir: ən vacibi əlbəttə ki, Keras-dır, lakin o, həmçinin məlumat yükləmə və preprocessing opsları (tf.data, [tf.io](https://www.tensorflow.org/tf.io) və s.), şəkil emalı opsları (tf.image), signal emalı opsları (tf.signal) və daha çoxuna

malikdir (TensorFlow-un Python API-si haqqında ümumi baxış üçün Şəkil 12-1-ə baxın).

İPUCU

TensorFlow API-nin bir çox paket və funksiyalarını əhatə edəcəyik, lakin hamısını əhatə etmək mümkün deyil, buna görə siz həqiqətən API-ni nəzərdən keçirməyə vaxt ayırmalısınız; onun olduqca zəngin və yaxşı sənədləşdirilmiş olduğunu görəcəksiniz.

Ən aşağı səviyyədə, hər bir TensorFlow əməliyyatı (qısaca *op*) yüksək səmərəli C++ kodu ilə tətbiq edilir. Bir çox əməliyyatların *kernels* (nüvələr) adlanan çoxsaylı tətbiqləri var: hər bir kernel müəyyən bir cihaz növünə, məsələn, CPU, GPU və ya hətta TPU (tensor processing units) kimi xüsusi cihazlara həsr edilmişdir. Bildiyiniz kimi, GPU-lar hesablamaları çoxsaylı kiçik hissələrə bölmək və onları çoxsaylı GPU threads üzərində paralel işə salmaqla hesablamaları dramatik şəkildə sürətləndirə bilər. TPU-lar daha da sürətlidir: onlar xüsusilə deep learning əməliyyatları üçün hazırlanmış xüsusi ASIC çipləridir (TensorFlow-u GPU və ya TPU-larla necə istifadə edəcəyimizi Fəsil 19-da müzakirə edəcəyik).

Şəkil 12-1. TensorFlow-un Python API-si

TensorFlow-un arxitekturası Şəkil 12-2-də göstərilmişdir. Kodunuzun əksər hissəsi yüksək səviyyəli API-lərdən (xüsusilə Keras və tf.data) istifadə edəcək, lakin daha çox çevikliyə ehtiyacınız olduqda aşağı səviyyəli Python API-dən istifadə edəcək, tensorlar ilə birbaşa işləyəcəksiniz. İstənilən halda, TensorFlow-un icra mühərriki

əməliyyatları səmərəli şəkildə, hətta çoxsaylı cihazlar və maşınlar arasında (əgər siz ona desəniz) işə salmağın qayğısına qalacaq.

TensorFlow yalnız Windows, Linux və macOS-da deyil, həmçinin mobil cihazlarda (*TensorFlow Lite* istifadə edərək), iOS və Android də daxil olmaqla, işləyir (bax Fəsil 19). Qeyd edək ki, Python API-sindən istifadə etmək istəməsəniz, digər dillər üçün API-lər də mövcuddur: C++, Java və Swift API-ləri var. Hətta *TensorFlow.js* adlı JavaScript tətbiqi var ki, modelinizi birbaşa brauzerdə işə salmağa imkan verir.

Şəkil 12-2. TensorFlow-un arxitekturası

TensorFlow kitabxanadan daha çox şeydir. TensorFlow geniş bir ekosistemin mərkəzindədir. Əvvəlcə, vizuallaşdırma üçün TensorBoard var (bax Fəsil 10). Sonra, TensorFlow Extended (TFX) var ki, bu Google tərəfindən TensorFlow layihələrini istehsalata çıxarmaq üçün qurulmuş kitabxanalar toplusudur: məlumat doğrulama, preprocessing, model analizi və xidmət göstərmə (TF Serving ilə; bax Fəsil 19) alətlərini ehtiva edir. Google-un *TensorFlow Hub* -ı əvvəlcədən öyrədilmiş neyron şəbəkələrini asanlıqla yükləmək və təkrar istifadə etmək üçün bir yol təqdim edir. Siz həmçinin TensorFlow-un model bağçasında bir çox neyron şəbəkə arxitekturasını, bəziləri əvvəlcədən öyrədilmiş şəkildə əldə edə bilərsiniz. Daha çox TensorFlow əsaslı layihələr üçün TensorFlow Resources və <https://github.com/jtoy/awesome-tensorflow> saytına baxın. GitHub-da yüzlərlə TensorFlow layihəsi tapa bilərsiniz, buna görə nə etmək istəsəniz, çox vaxt mövcud kodu tapmaq asandır.

İPUCU

Getdikcə daha çox ML məqalələri tətbiqləri ilə birlikdə, bəzən hətta əvvəlcədən öyrədilmiş modellərlə nəşr olunur. Onları asanlıqla tapmaq üçün <https://paperswithcode.com> saytına baxın.

Sonuncu, lakin ən az deyil, TensorFlow-un həvəsli və faydalı inkişaf etdiricilərdən ibarət xüsusi bir komandası, həmçinin onu təkmilləşdirməyə töhfə verən böyük bir icması var. Texniki suallar vermək üçün <https://stackoverflow.com> saytından istifadə edin və sualınızı *tensorflow* və *python* ilə etiketləyin. Səhvlər və xüsusiyyət tələbləri haqqında məlumat vermək üçün GitHub TensorFlow Forumundan istifadə edə bilərsiniz. Ümumi müzakirələr üçün TensorFlow Forumuna qoşulun.

Yaxşı, kodlaşdırmağa başlamaq vaxtıdır!

TensorFlow-un NumPy kimi istifadəsi

TensorFlow-un API-si *tensorlar* ətrafında fırlanır, onlar bir əməliyyatdan digərinə axır – buna görə Tensor *Flow* adı. Bir tensor NumPy ndarray-a çox bənzəyir: adətən çoxölçülü massivdir, lakin həmçinin skalyar (sadə dəyər, məsələn, 42) saxlaya bilər. Bu tensorlar xüsusi məsrəf funksiyaları, xüsusi metrikalar, xüsusi qatlar və daha çoxunu yaradarkən vacib olacaq, buna görə onları necə yaratmaq və idarə etmək lazım olduğunu görək.

Tensorlar və Əməliyyatlar

`tf.constant()` ilə bir tensor yarada bilərsiniz. Məsələn, iki sıra və üç sütunlu float matrisini təmsil edən bir tensor:

```
>>> import tensorflow as tf
>>> t = tf.constant([[1., 2., 3.], [4., 5., 6.]]) # matrix
>>> t
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)>
```

Bir ndarray kimi, `tf.Tensor`-ın `shape` və `data type (dtype)` var:

```
>>> t.shape
TensorShape([2, 3])
>>> t.dtype
```

```
tf.float32
```

İndeksləmə NumPy-də olduğu kimi işləyir:

```
>>> t[:, 1:]
<tf.Tensor: shape=(2, 2), dtype=float32, numpy=
array([[2., 3.],
       [5., 6.]], dtype=float32)>
>>> t[..., 1, tf.newaxis]
<tf.Tensor: shape=(2, 1), dtype=float32, numpy=
array([[2.],
       [5.]], dtype=float32)>
```

Ən əsası, hər cür tensor əməliyyatları mövcuddur:

```
>>> t + 10
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[11., 12., 13.],
       [14., 15., 16.]], dtype=float32)>
>>> tf.square(t)
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)>
>>> t @ tf.transpose(t)
<tf.Tensor: shape=(2, 2), dtype=float32, numpy=
array([[14., 32.],
       [32., 77.]], dtype=float32)>
```

Qeyd edək ki, `t + 10` yazmaq `tf.add(t, 10)` çağırmağa ekvivalentdir (həqiqətən də, Python `t.__add__(10)` sehrli metodunu çağırır, bu da sadəcə `tf.add(t, 10)`-u çağırır). `-` və `*` kimi digər operatorlar da dəstəklənir. `@` operatoru Python 3.5-də matris vurması üçün əlavə edilmişdir: o, `tf.matmul()` funksiyasını çağırmağa ekvivalentdir.

QEYD

Bir çox funksiya və siniflərin aliasları var. Məsələn, `tf.add()` və `tf.math.add()` eyni funksiyadır. Bu, TensorFlow-a ən çox yayılmış əməliyyatlar üçün qısa adlar saxlamağa imkan verir, eyni zamanda yaxşı təşkil edilmiş paketləri qoruyur.

Bir tensor həmçinin skalyar dəyər saxlaya bilər. Bu halda, forma boşdur:

```
>>> tf.constant(42)
```

```
<tf.Tensor: shape=(), dtype=int32, numpy=42>
```

QEYD

Keras API-si özünün aşağı səviyyəli API-sinə malikdir, `tf.keras.backend`-də yerləşir. Bu paket adətən qısalıq üçün `K` kimi idxal edilir. O, əvvəllər `K.square()`, `K.exp()` və `K.sqrt()` kimi funksiyaları ehtiva edirdi, bunlara mövcud kodda rast gələ bilərsiniz: bu, Keras çoxsaylı backends dəstəklədiyi zaman portativ kod yazmaq üçün faydalı idi, lakin indi Keras yalnız TensorFlow-a məxsus olduğundan, TensorFlow-un aşağı səviyyəli API-sini birbaşa çağırmalısınız (məsələn, `K.square()` əvəzinə `tf.square()`). Texniki olaraq `K.square()` və onun dostları geri uyğunluq üçün hələ də mövcuddur, lakin `tf.keras.backend`

paketinin sənədləri yalnız bir neçə yardımçı funksiyanı, məsələn, `clear_session()`-ı (Fəsil 10-da qeyd edilmişdir) sadalayır.

Lazım olan bütün əsas riyazi əməliyyatları (`tf.add()`, `tf.multiply()`, `tf.square()`, `tf.exp()`, `tf.sqrt()` və s.) və NumPy-də tapa biləcəyiniz əksər əməliyyatları (məsələn, `tf.reshape()`, `tf.squeeze()`, `tf.tile()`) tapa bilərsiniz. Bəzi funksiyalar NumPy-dəkindən fərqli ada malikdir; məsələn, `tf.reduce_mean()`, `tf.reduce_sum()`, `tf.reduce_max()` və `tf.math.log()` müvafiq olaraq `np.mean()`, `np.sum()`, `np.max()` və `np.log()`-un ekvivalentidir. Ad fərqli olduqda, bunun üçün adətən yaxşı bir səbəb var. Məsələn, TensorFlow-da `tf.transpose(t)` yazmalısınız; NumPy-dəki kimi `t.T` yazma bilməzsiniz. Səbəb odur ki, `tf.transpose()` funksiyası NumPy-in `T` atributu ilə tam eyni şeyi etmir: TensorFlow-da, transpozisiya edilmiş məlumatların öz surəti ilə yeni bir tensor yaradılır, NumPy-də isə `t.T` eyni məlumat üzərində transpozisiya edilmiş bir görünüşdür. Eynilə, `tf.reduce_sum()` əməliyyatı bu adla adlandırılmışdır, çünki onun GPU kernel (yəni, GPU tətbiqi) elementlərin əlavə edilməsi sırasına zəmanət verməyən bir reduce alqoritmindən istifadə edir: 32-bit float-ların məhdud dəqiqliyi səbəbindən, bu əməliyyatı hər çağırduğunuzda nəticə çox cüzi dəyişə bilər. Eyni şey `tf.reduce_mean()` üçün də doğrudur (lakin təbii ki, `tf.reduce_max()` deterministikdir).

Tensorlar və NumPy

Tensorlar NumPy ilə yaxşı işləyir: siz NumPy massivindən tensor yarada bilərsiniz və əksinə. Siz hətta NumPy massivlərinə TensorFlow əməliyyatlarını və tensorlara NumPy əməliyyatlarını tətbiq edə bilərsiniz:

```
>>> import numpy as np
>>> a = np.array([2., 4., 5.])
```



```
>>> tf.constant(a)
<tf.Tensor: id=111, shape=(3,), dtype=float64, numpy=array([2., 4., 5.])>
>>> t.numpy() # or np.array(t)
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)
>>> tf.square(a)
<tf.Tensor: id=116, shape=(3,), dtype=float64, numpy=array([4., 16., 25.])>
>>> np.square(t)
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)
```

XƏBƏRDARLIQ

Diqqət yetirin ki, NumPy standart olaraq 64-bit dəqiqlikdən istifadə edir, TensorFlow isə 32-bit. Bunun səbəbi, 32-bit dəqiqliyinin adətən neyron şəbəkələr üçün kifayət qədər çox olması, üstəlik daha sürətli işləməsi və daha az RAM istifadə etməsidir. Buna görə NumPy massivindən tensor yaratdıqda, `dtype=tf.float32` təyin etdiyinizə əmin olun.

Tip Çevirmələri

Tip çevirmələri performansla əhəmiyyətli dərəcədə zərər verə bilər və avtomatik olaraq edildikdə asanlıqla diqqətdən qaça bilər. Bunun qarşısını almaq üçün, TensorFlow heç bir avtomatik tip çevirməsi həyata keçirmir: uyğun olmayan tipli tensorlarda əməliyyat yerinə yetirməyə çalışsanız, sadəcə olaraq istisna (exception) yaradır. Məsələn, float tensor ilə integer tensor əlavə edə bilməzsiniz, hətta 32-bit float və 64-bit float əlavə edə bilməzsiniz:

```
>>> tf.constant(2.) + tf.constant(40)
[...] InvalidArgumentError: [...] expected to be a float tensor [...]
>>> tf.constant(2.) + tf.constant(40., dtype=tf.float64)
```

```
[...] InvalidArgumentError: [...] expected to be a float tensor  
[...]
```

Bu, əvvəlcə bir az narahat ola bilər, lakin yaxşı bir səbəb üçün olduğunu xatırlayın! Və təbii ki, həqiqətən tip çevirmək lazım olduqda `tf.cast()` istifadə edə bilərsiniz:

```
>>> t2 = tf.constant(40., dtype=tf.float64)  
>>> tf.constant(2.0) + tf.cast(t2, tf.float32)  
  
<tf.Tensor: id=136, shape=(), dtype=float32, numpy=42.0>
```

Dəyişənlər (Variables)

İndiyə qədər gördüyümüz `tf.Tensor` dəyərləri dəyişməzdir: biz onları dəyişdirə bilmirik. Bu o deməkdir ki, neyron şəbəkədə çəkirləri tətbiq etmək üçün adi tensorlardan istifadə edə bilmirik, çünki onlar backpropagation tərəfindən dəyişdirilməlidir. Üstəlik, digər parametrlər zamanla dəyişə bilər (məsələn, momentum optimizator keçmiş qradientləri izləyir). Bizə lazım olan `tf.Variable`-dir:

```
>>> v = tf.Variable([[1., 2., 3.], [4., 5., 6.]])  
>>> v  
<tf.Variable 'Variable:0' shape=(2, 3) dtype=float32, numpy=  
array([[1., 2., 3.],  
       [4., 5., 6.]], dtype=float32)>
```

`tf.Variable` `tf.Tensor` kimi davranır: onunla eyni əməliyyatları yerinə yetirə bilərsiniz, NumPy ilə yaxşı işləyir və tiplərlə eyni dərəcədə seçicidir. Lakin o, `assign()` metodu (və ya `assign_add()` və ya `assign_sub()`), verilmiş dəyərlə dəyişəni artırır və ya azaldır) ilə yerində dəyişdirilə bilər. Siz həmçinin fərdi hüceyrələri (və ya dilimləri) onların `assign()` metodu

ilə və ya `scatter_update()` və ya `scatter_nd_update()` metodları ilə dəyişə bilərsiniz:

```
v.assign(2 * v) # v indi [[2., 4., 6.], [8., 10., 12.]]-yə bərabərdir
v[0, 1].assign(42) # v indi [[2., 42., 6.], [8., 10., 12.]]-yə bərabərdir
v[:, 2].assign([0., 1.]) # v indi [[2., 42., 0.], [8., 10., 1.]]-yə
bərabərdir
v.scatter_nd_update( # v indi [[100., 42., 0.], [8., 10., 200.]]-yə
bərabərdir
    indices=[[0, 0], [1, 2]],
    updates=[100., 200.])
```

Birbaşa təyinat işləməyəcək:

```
>>> v[1] = [7., 8., 9.]

[...] TypeError: 'ResourceVariable' object does not support item
assignment
```

QEYD

Praktikada dəyişənləri əl ilə yaratmaq nadir hallarda lazım olacaq; Keras `add_weight()` metodu təqdim edir ki, o, bunun qayğısına qalacaq, görəcəksiniz. Üstəlik, model parametrləri ümumiyyətlə optimizatorlar tərəfindən birbaşa yenilənəcək, buna görə dəyişənləri əl ilə yeniləmək nadir hallarda lazım olacaq.

Digər Məlumat Strukturları

TensorFlow bir neçə digər məlumat strukturunu dəstəkləyir, o cümlədən (daha çox təfərrüat üçün bu fəslin notebook-unun "Other Data Structures" bölməsinə və ya Əlavə C-yə baxın):

Sparse tensors (tf.SparseTensor)

Əksəriyyəti sıfır olan tensorları səmərəli şəkildə təmsil edir. `tf.sparse` paketi seyrək tensorlar üçün əməliyyatları ehtiva edir.

Tensor arrays (`tf.TensorArray`)

Tensorların siyahılarıdır. Onlar standart olaraq sabit uzunluqludur, lakin istəyə bağlı olaraq genişləndirilə bilər. Tərkibindəki bütün tensorlar eyni forma və data type-a malik olmalıdır.

Ragged tensors (`tf.RaggedTensor`)

Hamısı eyni rank və data type-a malik, lakin müxtəlif ölçülü tensorların siyahılarını təmsil edir. Tensor ölçülərinin dəyişdiyi ölçülər *ragged dimensions* adlanır. `tf.ragged` paketi ragged tensorlar üçün əməliyyatları ehtiva edir.

String tensors

`tf.string` tipli adi tensorlardır. Bunlar byte stringləri təmsil edir, Unicode stringlər deyil, buna görə bir Unicode string (məsələn, adi Python 3 stringi "café") istifadə edərək string tensor yaratsanız, o, avtomatik olaraq UTF-8-ə kodlaşdırılacaq (məsələn, `b"caf\xc3\xa9"`). Alternativ olaraq, Unicode stringləri `tf.int32` tipli tensorlardan istifadə edərək təmsil edə bilərsiniz, burada hər bir element bir Unicode kod nöqtəsini təmsil edir (məsələn, `[99, 97, 102, 233]`). `tf.strings` paketi (sonunda `s` var) byte stringlər və Unicode stringlər üçün əməliyyatları (və birini digərinə çevirmək üçün) ehtiva edir. Qeyd etmək vacibdir ki, `tf.string` atomikdir, yəni onun uzunluğu tensorun formasında görünür. Onu Unicode tensoruna (yəni Unicode kod nöqtələrini saxlayan `tf.int32` tipli tensor) çevirdikdən sonra uzunluq formada görünür.

Sets (Çoxluqlar)

Adi tensorlar (və ya seyrək tensorlar) kimi təmsil olunur. Məsələn, `tf.constant([[1, 2], [3, 4]])` iki çoxluğu $\{1, 2\}$ və $\{3, 4\}$ təmsil edir. Ümumiyyətlə, hər bir çoxluq tensorun son oxundakı bir vektorla təmsil olunur. `tf.sets` paketindəki əməliyyatlardan istifadə edərək çoxluqları idarə edə bilərsiniz.

Queues (Növbələr)

Tensorları çoxsaylı addımlar boyunca saxlayır. TensorFlow müxtəlif növ növbələr təklif edir: əsas first-in, first-out (FIFO) növbələri (`FIFOQueue`), əlavə olaraq bəzi elementlərə üstünlük verə bilən növbələr (`PriorityQueue`), elementləri qarışdıran növbələr (`RandomShuffleQueue`) və müxtəlif formalı elementləri padding ilə yığan növbələr (`PaddingFIFOQueue`). Bu siniflərin hamısı `tf.queue` paketindədir.

Tensorlar, əməliyyatlar, dəyişənlər və müxtəlif məlumat strukturları ilə silahlanaraq, indi modellərinizi və təlim alqoritmlərinizi fərdiləşdirməyə hazırsınız!

Modellərin və Təlim Alqoritmlərinin Fərdiləşdirilməsi

Siz xüsusi loss funksiyası yaratmaqla başlayacaqsınız, bu, sadə və geniş yayılmış bir istifadə halıdır.

Xüsusi Loss Funksiyaları

Tutaq ki, regression modeli öyrətmək istəyirsiniz, lakin təlim setiniz bir az səs-küylüdür. Əlbəttə, siz əvvəlcə dataset-i təmizləməyə və ya outlier-ləri çıxarmağa və ya düzəltməyə çalışmaqla başlayırsınız, lakin bu

kifayət etmir; dataset hələ də səs-küylüdür. Hansı loss funksiyasından istifadə etməlisiniz? Mean squared error böyük səhvləri çox cəzalandıra və modelinizin qeyri-dəqiq olmasına səbəb ola bilər. Mean absolute error outlier-ləri o qədər də cəzalandırmaz, lakin təlim yığılmaq üçün bir müddət çəkə bilər və öyrədilmiş model çox dəqiq olmaya bilər. Bu, yəqin ki, köhnə MSE əvəzinə Huber loss (Fəsil 10-da təqdim edilmişdir) istifadə etmək üçün yaxşı bir vaxtdır. Huber loss Keras-da mövcuddur (sadəcə `tf.keras.losses.Huber` sinfinin bir nümunəsini istifadə edin), lakin gəlin fərz edək ki, o, mövcud deyil. Onu tətbiq etmək üçün, etiketləri və modelin proqnozlarını argument kimi qəbul edən və TensorFlow əməliyyatlarından istifadə edərək bütün lossları (hər nümunə üçün bir) ehtiva edən bir tensor hesablayan bir funksiya yaradın:

```
def huber_fn(y_true, y_pred):  
    error = y_true - y_pred  
    is_small_error = tf.abs(error) < 1  
    squared_loss = tf.square(error) / 2  
    linear_loss = tf.abs(error) - 0.5  
  
    return tf.where(is_small_error, squared_loss, linear_loss)
```

XƏBƏRDARLIQ

Daha yaxşı performans üçün, bu nümunədə olduğu kimi vektorlaşdırılmış (vectorized) tətbiqdən istifadə etməlisiniz. Üstəlik, TensorFlow-un qrafik optimallaşdırma xüsusiyyətlərindən faydalanmaq istəyirsinizsə, yalnız TensorFlow əməliyyatlarından istifadə etməlisiniz.

Fərdi lossların ortalamasını qaytarmaq da mümkündür, lakin bu tövsiyə edilmir, çünki bu, lazım olduqda class weights və ya sample weights istifadə etməyi mümkünsüz edir (bax Fəsil 10).

İndi Keras modelini kompilyasiya edərkən bu Huber loss funksiyasından istifadə edə bilərsiniz, sonra modeli həmişəki kimi öyrədə bilərsiniz:

```
model.compile(loss=huber_fn, optimizer="nadam")
```

```
model.fit(X_train, y_train, [...])
```

Və hamısı budur! Təlim zamanı hər bir batch üçün Keras loss-u hesablamaq üçün `huber_fn()` funksiyasını çağıracaq, sonra loss-un bütün model parametrlərinə görə qradientlərini hesablamaq üçün reverse-mode autodiff istifadə edəcək və nəhayət gradient descent addımını yerinə yetirəcək (bu nümunədə Nadam optimizatorundan istifadə etməklə). Üstəlik, o, dövrün əvvəlindən ümumi loss-u izləyəcək və ortalama loss-u göstərəcək.

Bəs bu xüsusi loss modeli saxladığınız zaman nə baş verir?

Xüsusi Komponentləri Olan Modellərin Saxlanması və Yüklənməsi

Xüsusi loss funksiyasını ehtiva edən bir modeli saxlamaq yaxşı işləyir, lakin onu yüklədikdə, funksiya adını faktiki funksiyaya xəritələşdirən bir lüğət təqdim etməli olacaqsınız. Daha ümumi olaraq, xüsusi obyektləri ehtiva edən bir modeli yüklədikdə, adları obyektlərə xəritələşdirməlisiniz:

```
model = tf.keras.models.load_model("my_model_with_a_custom_loss",  
                                   custom_objects={"huber_fn":  
                                                   huber_fn})
```

İPUCU

Əgər `huber_fn()` funksiyasını `@keras.utils.register_keras_serializable()` ilə bəzəsəniz, o, avtomatik olaraq `load_model()` funksiyası üçün əlçatan olacaq: onu `custom_objects` lüğətinə daxil etməyə ehtiyac yoxdur.

Cari tətbiqlə, -1 ilə 1 arasındakı hər hansı bir səhv "kiçik" sayılır. Bəs fərqli bir hədd (threshold) istəyirsinizsə? Bir həll, konfigurasiya edilmiş loss funksiyasını yaradan bir funksiya yaratmaqdır:

```
def create_huber(threshold=1.0):
    def huber_fn(y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < threshold
        squared_loss = tf.square(error) / 2
        linear_loss = threshold * tf.abs(error) - threshold ** 2 / 2
        return tf.where(is_small_error, squared_loss, linear_loss)
    return huber_fn
```

```
model.compile(loss=create_huber(2.0), optimizer="nadam")
```

Təəssüf ki, modeli saxladıqda, threshold saxlanılmayacaq. Bu o deməkdir ki, modeli yükləyərkən threshold dəyərini təyin etməli olacaqsınız (qeyd edək ki, istifadə edəcəyiniz ad "huber_fn"-dir, bu, Keras-a verdiyiniz funksiyanın adıdır, onu yaradan funksiyanın adı deyil):

```
model = tf.keras.models.load_model(
    "my_model_with_a_custom_loss_threshold_2",
    custom_objects={"huber_fn": create_huber(2.0)})
```

Bunu həll etmək üçün `tf.keras.losses.Loss` sinfinin alt sinfini yarada və onun `get_config()` metodunu tətbiq edə bilərsiniz:

```
class HuberLoss(tf.keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)
```



```

def call(self, y_true, y_pred):
    error = y_true - y_pred
    is_small_error = tf.abs(error) < self.threshold
    squared_loss = tf.square(error) / 2
    linear_loss = self.threshold * tf.abs(error) - self.threshold**2 /

    return tf.where(is_small_error, squared_loss, linear_loss)
def get_config(self):
    base_config = super().get_config()

    return {**base_config, "threshold": self.threshold}

```

Bu kodu nəzərdən keçirək:

- Konstruktor `**kwargs` qəbul edir və onları ana konstruktora ötürür, bu da standart hiperparametrləri idarə edir: `loss`-un adı və fərdi instans `loss`larını birləşdirmək üçün istifadə olunan reduksiya alqoritmi. Standart olaraq bu "AUTO"-dur, "SUM_OVER_BATCH_SIZE"-a ekvivalentdir: `loss`, `sample weights` varsa, onlarla çəkilmiş instans `loss`larının cəmi olacaq və `batch ölçüsünə` bölünəcək (çəkilərin cəminə yox, buna görə də bu *çəkilmiş ortalama* deyil). Digər mümkün dəyərlər "SUM" və "NONE"-dur.
- `call()` metodu etikətləri və proqnozları qəbul edir, bütün instans `loss`larını hesablayır və onları qaytarır.
- `get_config()` metodu hər bir hiperparametr adını onun dəyərinə xəritələşdirən lüğəti qaytarır. O, əvvəlcə ana sinfin `get_config()` metodunu çağırır, sonra yeni hiperparametrləri bu lüğətə əlavə edir.

Sonra modeli kompilyasiya edərkən bu sinfin istənilən nümunəsindən istifadə edə bilərsiniz:

```
model.compile(loss=HuberLoss(2.), optimizer="nadam")
```

Modeli saxladıqda, `threshold` onunla birlikdə saxlanılacaq; modeli yüklədikdə isə, sinif adını sinfin özünə xəritələşdirməlisiniz:

```
model = tf.keras.models.load_model("my_model_with_a_custom_loss_class",
```

```
custom_objects={"HuberLoss":  
HuberLoss})
```

Modeli saxladıqda, Keras loss nümunəsinin `get_config()` metodunu çağırır və konfigurasiyanı SavedModel formatında saxlayır. Modeli yüklədikdə, o, HuberLoss sinfində `from_config()` sinif metodunu çağırır: bu metod əsas sinif (Loss) tərəfindən tətbiq edilir və sinfin nümunəsini yaradır, `**config`-i konstruktora ötürür.

Losslar üçün hamısı budur! İndi görəcəyiniz kimi, xüsusi aktivasiya funksiyaları, ilkinləşdiricilər, regularizatorlar və məhdudiyyətlər də çox fərqli deyil.

Xüsusi Aktivasiya Funksiyaları, İlkinləşdiricilər, Regularizatorlar və Məhdudiyyətlər

Keras funksiyalarının əksəriyyəti, məsələn, losslar, regularizatorlar, məhdudiyyətlər, ilkinləşdiricilər, metrikalar, aktivasiya funksiyaları, qatlar və hətta tam modellər eyni şəkildə fərdiləşdirilə bilər. Əksər hallarda, müvafiq giriş və çıxışları olan sadə bir funksiya yazmaq kifayətdir. Burada xüsusi aktivasiya funksiyasının (`tf.keras.activations.softplus()` və ya `tf.nn.softplus()`-a ekvivalent), xüsusi Glorot ilkinləşdiricisinin (`tf.keras.initializers.glorot_normal()`-a ekvivalent), xüsusi ℓ_1 regularizatorunun (`tf.keras.regularizers.l1(0.01)`-a ekvivalent) və çəkilərin hamısının müsbət olmasını təmin edən xüsusi məhdudiyyətin (`tf.keras.constraints.nonneg()` və ya `tf.nn.relu()`-ya ekvivalent) nümunələri verilmişdir:

```
def my_softplus(z):
```

```

    return tf.math.log(1.0 + tf.exp(z))

def my_glorot_initializer(shape, dtype=tf.float32):
    stddev = tf.sqrt(2. / (shape[0] + shape[1]))
    return tf.random.normal(shape, stddev=stddev, dtype=dtype)

def my_l1_regularizer(weights):
    return tf.reduce_sum(tf.abs(0.01 * weights))

def my_positive_weights(weights):          # return value is just
tf.nn.relu(weights)

    return tf.where(weights < 0., tf.zeros_like(weights),
weights)

```

Gördüyünüz kimi, argumentlər xüsusi funksiyanın növündən asılıdır. Bu xüsusi funksiyalar daha sonra adi qaydada istifadə edilə bilər:

```

layer = tf.keras.layers.Dense(1, activation=my_softplus,
                                kernel_initializer=my_glorot_initializer,
                                kernel_regularizer=my_l1_regularizer,

                                kernel_constraint=my_positive_weights)

```

Aktivasiya funksiyası bu Dense qatının çıxışına tətbiq ediləcək və onun nəticəsi növbəti qata ötürüləcək. Qatın çəkilişləri ilkinləşdirici tərəfindən qaytarılan dəyərdən istifadə edərək ilkinləşdiriləcək. Hər bir təlim addımında çəkilişlər regularizasiya loss-unu hesablamaq üçün regularizasiya funksiyasına ötürüləcək, bu da əsas loss-a əlavə edilərək təlim üçün istifadə olunan son loss-u əmələ gətirəcək. Nəhayət, məhdudiyyət funksiyası hər təlim addımından sonra çağırılacaq və qatın çəkilişləri məhdudlaşdırılmış çəkilişlə əvəz ediləcək.

Əgər bir funksiyanın model ilə birlikdə saxlanmalı olan hiperparametrləri varsa, o zaman müvafiq sinfi, məsələn, `tf.keras.regularizers.Regularizer`,

`tf.keras.constraints.Constraint`,
`tf.keras.initializers.Initializer`, və ya `tf.keras.layers.Layer`
(hər hansı bir qat, o cümlədən aktivasiya funksiyaları üçün) alt sinfinə
çevirmək istəyəcəksiniz. Xüsusi loss üçün etdiyiniz kimi, burada ℓ_1
regularizasiyası üçün onun `factor` hiperparametrini saxlayan sadə bir
sınıf nümunəsi verilmişdir (bu dəfə ana konstruktoru və ya
`get_config()` metodunu çağırmağa ehtiyac yoxdur, çünki bunlar ana
sınıf tərəfindən müəyyən edilməmişdir):

```
class MyL1Regularizer(tf.keras.regularizers.Regularizer):  
    def __init__(self, factor):  
        self.factor = factor  
    def __call__(self, weights):  
        return tf.reduce_sum(tf.abs(self.factor * weights))  
    def get_config(self):  
  
        return {"factor": self.factor}
```

Qeyd edək ki, lossar, qatlar (aktivasiya funksiyaları daxil) və modellər
üçün `call()` metodunu, regularizatorlar, ilkinləşdiricilər və
məhdudiyyətlər üçün isə `__call__()` metodunu tətbiq etməlisiniz.
Metrikalar üçün işlər bir az fərqlidir, indi görəcəksiniz.

Xüsusi Metrikalar

Lossar və metrikalar konseptual olaraq eyni şey deyil: lossar (məsələn,
cross entropy) gradient descent tərəfindən modeli *təlim etmək* üçün
istifadə olunur, buna görə onlar diferensiallana bilən olmalıdır (heç
olmasa qiymətləndirildikləri nöqtələrdə) və onların qradientləri heç bir
yerdə sıfır olmamalıdır. Üstəlik, onların insanlar tərəfindən asanlıqla
interpretasiya edilməsi vacib deyil. Bunun əksinə, metrikalar (məsələn,
dəqiqlik) modeli *qiymətləndirmək* üçün istifadə olunur: onlar daha asan

interpretasiya edilməlidir və differensialana bilən olmaya və ya hər yerdə sıfır qradiyentə malik ola bilərlər.

Bununla belə, əksər hallarda, xüsusi metrika funksiyasını təyin etmək xüsusi loss funksiyasını təyin etməklə eynidir. Əslində, biz əvvəllər yaratdığımız Huber loss funksiyasından metrika kimi də istifadə edə bilərik; o, yaxşı işləyəcək (və davamlılıq da eyni şəkildə işləyəcək, bu halda yalnız funksiyanın adını saxlayacaq, "huber_fn", threshold-u yox):

```
model.compile(loss="mse", optimizer="nadam",  
metrics=[create_huber(2.0)])
```

Təlim zamanı hər bir batch üçün Keras bu metrikanı hesablayacaq və dövrün əvvəlindən onun ortalamasını izləyəcək. Əksər hallarda, bu, tam olaraq istədiyiniz şeydir. Lakin həmişə deyil! Məsələn, ikili təsnifatçının precision-ını nəzərdən keçirək. Fəsil 3-də gördüyünüz kimi, precision, true positive-lərin sayının müsbət proqnozların ümumi sayına (həm true positive, həm də false positive daxil olmaqla) bölünməsidir. Tutaq ki, model birinci batch-də beş müsbət proqnoz verdi, onlardan dördü doğru idi: bu, 80% precision-dır. Sonra fərz edək ki, model ikinci batch-də üç müsbət proqnoz verdi, lakin hamısı səhv idi: bu, ikinci batch üçün 0% precision-dır. Əgər bu iki precision-un ortalamasını hesablasanız, 40% alırsınız. Amma bir dəqiqə gözləyin – bu, modelin bu iki batch üzərindəki precision-ı *deyil*! Həqiqətən də, səkkiz müsbət proqnozdan (5 + 3) cəmi dörd true positive (4 + 0) var idi, buna görə ümumi precision 50%-dir, 40% deyil. Bizə lazım olan, true positive-lərin sayını və false positive-lərin sayını izləyə bilən və tələb olunduqda bu rəqəmlərə əsaslanan precision-u hesablaya bilən bir obyektidir. `tf.keras.metrics.Precision` sinfi məhz bunu edir:

```
>>> precision = tf.keras.metrics.Precision()
>>> precision([0, 1, 1, 1, 0, 1, 0, 1], [1, 1, 0, 1, 0, 1, 0, 1])
<tf.Tensor: shape=(), dtype=float32, numpy=0.8>
>>> precision([0, 1, 0, 0, 1, 0, 1, 1], [1, 0, 1, 1, 0, 0, 0, 0])
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=0.5>
```

Bu nümunədə, biz bir Precision obyektini yaratdıq, sonra onu funksiya kimi istifadə edərək, birinci batch üçün etiketləri və proqnozları, sonra ikinci batch üçün ötürdük (istəsəniz, sample weights da ötürə bilərsiniz). Biz indi müzakirə etdiyimiz nümunədəki kimi eyni sayda true və false positive istifadə etdik. Birinci batch-dən sonra 80% precision qaytarır; sonra ikinci batch-dən sonra 50% qaytarır (bu, indiyə qədərki ümumi precision-dır, ikinci batch-in precision-ı deyil). Buna *streaming metric* (və ya *stateful metric*) deyilir, çünki o, batch-batch tədricən yenilənir.

İstənilən vaxt biz `result()` metodunu çağıraraq metrikanın cari dəyərini əldə edə bilərik. Biz həmçinin `variables` atributundan istifadə edərək onun dəyişənlərinə (true və false positive-lərin sayını izləyən) baxa və `reset_states()` metodundan istifadə edərək bu dəyişənləri sıfırlaya bilərik:

```
>>> precision.result()
<tf.Tensor: shape=(), dtype=float32, numpy=0.5>
>>> precision.variables
[<tf.Variable 'true_positives:0' [...], numpy=array([4.], dtype=float32)>,
 <tf.Variable 'false_positives:0' [...], numpy=array([4.], dtype=float32)>]
```

```
>>> precision.reset_states() # hər iki dəyişən 0.0-a sıfırlanır
```

Əgər öz xüsusi streaming metrika funksiyanızı təyin etməlisinizsə, `tf.keras.metrics.Metric` sinfinin alt sinfini yaradın. Burada ümumi Huber loss-u və indiyə qədər görülmüş instansların sayını izləyən əsas bir nümunə verilmişdir. Nəticə tələb olunduqda, nisbəti qaytarır, bu da sadəcə ortalama Huber loss-dur:

```

class HuberMetric(tf.keras.metrics.Metric):
    def __init__(self, threshold=1.0, **kwargs):
        super().__init__(**kwargs) # handles base args (e.g., dtype)
        self.threshold = threshold
        self.huber_fn = create_huber(threshold)
        self.total = self.add_weight("total", initializer="zeros")
        self.count = self.add_weight("count", initializer="zeros")
    def update_state(self, y_true, y_pred, sample_weight=None):
        sample_metrics = self.huber_fn(y_true, y_pred)
        self.total.assign_add(tf.reduce_sum(sample_metrics))
        self.count.assign_add(tf.cast(tf.size(y_true), tf.float32))
    def result(self):
        return self.total / self.count
    def get_config(self):
        base_config = super().get_config()

        return {**base_config, "threshold": self.threshold}

```

Bu kodu nəzərdən keçirək:

- Konstruktor metrika vəziyyətini çoxsaylı batch-lər boyunca izləmək üçün lazım olan dəyişənləri yaratmaq üçün `add_weight()` metodundan istifadə edir – bu halda, bütün Huber loss-ların cəmi (`total`) və indiyə qədər görülmüş instansların sayı (`count`). İstəsəniz, dəyişənləri əl ilə də yarada bilərsiniz. Keras atribut kimi təyin edilmiş hər hansı `tf.Variable`-ı (və daha ümumi olaraq, hər hansı "trackable" obyekt, məsələn, qatlar və ya modellər) izləyir.
- `update_state()` metodu, bu sinfin nümunəsini funksiya kimi istifadə etdikdə çağırılır (Precision obyekt, ilə etdiyimiz kimi). O, bir batch üçün etikətlər və proqnozlar (və sample weights, lakin bu halda onları nəzər almırıq) verildikdə dəyişənləri yeniləyir.
- `result()` metodu son nəticəni hesablayır və qaytarır, bu halda bütün instanslar üzərində ortalama Huber metric-dir. Metrikə funksiya kimi istifadə etdikdə, əvvəlcə `update_state()` metodu çağırılır, sonra `result()` metodu çağırılır və onun çıxışı qaytarılır.
- Biz həmçinin `get_config()` metodunu tətbiq edirik ki, `threshold` model ilə birlikdə saxlanılsın.
- `reset_states()` metodunun defolt tətbiqi bütün dəyişənləri 0.0-a sıfırlayır (lakin lazım olduqda onu override edə bilərsiniz).

QEYD

Keras dəyişənlərin davamlılığını problemsiz idarə edəcək; heç bir əməliyyat tələb olunmur.

Sadə bir funksiya istifadə edərək metrika təyin etdikdə, Keras avtomatik olaraq onu hər bir batch üçün çağırır və hər dövr ərzində ortalamanı izləyir, eynilə bizim əl ilə etdiyimiz kimi. Beləliklə, HuberMetric sinifimizin yeganə üstünlüyü threshold-un saxlanacaq olmasıdır. Lakin təbii ki, precision kimi bəzi metrikalar sadəcə batch-lər üzərində ortalama alınır bilməz: belə hallarda, streaming metric tətbiq etməkdən başqa seçim yoxdur.

İndi streaming metric qurduğunuza görə, xüsusi qat qurmaq sizə asan görünəcək!

Xüsusi Qatlar

Bəzən TensorFlow-un defolt tətbiqini təqdim etmədiyi ekzotik bir qat ehtiva edən bir arxitektura qurmaq istəyə bilərsiniz. Və ya sadəcə çox təkrarlanan bir arxitektura qurmaq istəyə bilərsiniz, burada müəyyən bir qat bloku dəfələrlə təkrarlanır və hər bir bloku tək bir qat kimi qəbul etmək rahat olar. Belə hallar üçün xüsusi qat qurmaq istəyəcəksiniz.

Çəkisi olmayan bəzi qatlar var, məsələn, `tf.keras.layers.Flatten` və ya `tf.keras.layers.ReLU`. Əgər çəkisiz bir xüsusi qat yaratmaq istəyirsinizsə, ən sadə seçim bir funksiya yazmaq və onu

`tf.keras.layers.Lambda` qatına bükməkdir. Məsələn, aşağıdakı qat girişlərinə eksponensial funksiyanı tətbiq edəcək:

```
exponential_layer = tf.keras.layers.Lambda(lambda x: tf.exp(x))
```

Bu xüsusi qat daha sonra hər hansı digər qat kimi, sequential API, functional API və ya subclassing API istifadə edərək istifadə edilə bilər. Siz onu aktivasiya funksiyası kimi də istifadə edə bilərsiniz və ya `activation=tf.exp` istifadə edə bilərsiniz. Eksponensial qat bəzən regression modelinin çıxış qatında, proqnozlaşdırılacaq dəyərlər çox fərqli miqyaslara malik olduqda (məsələn, 0.001, 10., 1000.) istifadə olunur. Əslində, eksponensial funksiya Keras-da standart aktivasiya funksiyalarından biridir, buna görə sadəcə `activation="exponential"` istifadə edə bilərsiniz.

Təxmin etdiyiniz kimi, stateful (yəni, çəkili) xüsusi qat qurmaq üçün `tf.keras.layers.Layer` sinfinin alt sinfini yaratmalısınız. Məsələn, aşağıdakı sinif Dense qatının sadələşdirilmiş versiyasını tətbiq edir:

```
class MyDense(tf.keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = tf.keras.activations.get(activation)
    def build(self, batch_input_shape):
        self.kernel = self.add_weight(
            name="kernel",
            shape=[batch_input_shape[-1], self.units],
            initializer="glorot_normal")
        self.bias = self.add_weight(
            name="bias",
            shape=[self.units],
            initializer="zeros")
    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)
    def get_config(self):
        base_config = super().get_config()
```

```

return {**base_config,
        "units": self.units,
        "activation":
tf.keras.activations.serialize(self.activation)}

```

Bu kodu nəzərdən keçirək:

- Konstruktor bütün hiperparametrləri argument kimi qəbul edir (bu nümunədə `units` və `activation`), və vacib olaraq, həmçinin `**kwargs` argumentini qəbul edir. O, ana konstruktoru çağırır və `kwargs`-ı ona ötürür: bu, `input_shape`, `trainable` və `name` kimi standart argumentlərin qayğısına qalır. Sonra hiperparametrləri atribut kimi saxlayır, `activation` argumentini `tf.keras.activations.get()` funksiyasından istifadə edərək müvafiq aktivasiya funksiyasına çevirir (bu, funksiyaları, standart stringləri "relu" və ya "swish" kimi, və ya sadəcə None qəbul edir).
- `build()` metodunun rolu, hər bir çəki üçün `add_weight()` metodunu çağıraraq qatın dəyişənlərini yaratmaqdır. `build()` metodu qat ilk dəfə istifadə edildikdə çağırılır. O zaman Keras bu qatın girişlərinin formasını biləcək və onu `build()` metoduna ötürəcək, bu da bəzi çəkiləri yaratmaq üçün çox vaxt lazımdır. Məsələn, birləşmə çəkiləri matrisini (yəni, "kernel") yaratmaq üçün əvvəlki qatdakı neyronların sayını bilməliyik: bu, girişlərin son ölçüsünə uyğundur. `build()` metodunun sonunda (və yalnız sonunda) ana sinfin `build()` metodunu çağırmalısınız: bu, Keras-a qatın qurulduğunu bildirir (o, sadəcə `self.built = True` təyin edir).
- `call()` metodu istədiyiniz əməliyyatları yerinə yetirir. Bu halda, girişlər X və qatın kernel-i arasında matris vurmasını hesablayırıq, bias vektorunu əlavə edirik və nəticəyə aktivasiya funksiyasını tətbiq edirik, bu da qatın çıxışını verir.
- `get_config()` metodu əvvəlki xüsusi siniflərdə olduğu kimidir. Qeyd edək ki, biz `tf.keras.activations.serialize()` çağıraraq aktivasiya funksiyasının tam konfigurasiyasını saxlayırıq.

İndi `MyDense` qatını hər hansı digər qat kimi istifadə edə bilərsiniz!

QEYD

Keras avtomatik olaraq çıxış formasını təxmin edir, qat dinamik olduğu hallar istisna olmaqla (qısa müddətdə görəcəksiniz). Bu (nadir) halda, `compute_output_shape()` metodunu tətbiq etməlisiniz, o, `TensorShape` obyektini qaytarmalıdır.

Çoxlu girişi olan bir qat yaratmaq üçün (məsələn, `Concatenate`), `call()` metodunun argumenti bütün girişləri ehtiva edən bir tuple olmalıdır. Çoxlu çıxışı olan bir qat yaratmaq üçün, `call()` metodu çıxışların siyahısını qaytarmalıdır. Məsələn, aşağıdakı oyuncaq qat iki giriş qəbul edir və üç çıxış qaytarır:

```
class MyMultiLayer(tf.keras.layers.Layer):
    def call(self, X):
        X1, X2 = X

        return X1 + X2, X1 * X2, X1 / X2
```

Bu qat indi hər hansı digər qat kimi istifadə edilə bilər, lakin təbii ki, yalnız `functional` və `subclassing` API-lərindən istifadə etməklə, `sequential` API-dən yox (o, yalnız bir giriş və bir çıxışı olan qatları qəbul edir).

Əgər qatınız təlim və test zamanı fərqli davranışa malik olmalıdırsa (məsələn, `Dropout` və ya `BatchNormalization` qatları istifadə edirsə), onda `call()` metoduna bir `training` argumenti əlavə etməli və nə edəcəyinizi qərar vermək üçün bu argumentdən istifadə etməlisiniz. Məsələn, təlim zamanı `Gaussian səs-küy` əlavə edən (`regularizasiya` üçün), lakin test zamanı heç nə etməyən bir qat yaradaq (Keras-da eyni şeyi edən bir qat var, `tf.keras.layers.GaussianNoise`):

```
class MyGaussianNoise(tf.keras.layers.Layer):
    def __init__(self, stddev, **kwargs):
        super().__init__(**kwargs)
        self.stddev = stddev
    def call(self, X, training=False):
```

```

if training:
    noise = tf.random.normal(tf.shape(X), stddev=self.stddev)
    return X + noise
else:
    return X

```

Bununla, indi sizə lazım olan hər hansı xüsusi qatı qura bilərsiniz! İndi xüsusi modellərin necə yaradılacağına baxaq.

Xüsusi Modellər

Fəsil 10-da, subclassing API-ni müzakirə edərkən xüsusi model siniflərinin yaradılmasına baxdıq. Bu, sadədir: `tf.keras.Model` sinfinin alt sinfini yaradın, konstruktorda qatlar və dəyişənlər yaradın və modelin etməsini istədiyiniz hər şeyi etmək üçün `call()` metodunu tətbiq edin. Məsələn, Şəkil 12-3-də təsvir edilən modeli qurmaq istədiyimizi fərz edək.

Şəkil 12-3. Xüsusi model nümunəsi: skip connection olan xüsusi ResidualBlock qatını ehtiva edən ixtiyari bir model

Girişlər ilk sıx qatdan keçir, sonra iki sıx qat və bir əlavə əməliyyatından ibarət olan *residual block* (bax Fəsil 14, residual block öz girişlərini çıxışlarına əlavə edir) vasitəsilə, sonra eyni residual block-dan daha üç dəfə, sonra ikinci residual block-dan keçir və son nəticə sıx çıxış qatından keçir. Bu model məntiqli görünürsə, narahat olmayın; bu, istədiyiniz hər hansı bir modeli, hətta loop-lar və skip connections ehtiva edənləri asanlıqla qura biləcəyinizi nümayiş etdirmək üçün yalnız bir nümunədir. Bu modeli tətbiq etmək üçün, əvvəlcə `ResidualBlock` qatını yaratmaq daha yaxşıdır, çünki biz bir neçə eyni blok yaradacağıq (və bəlkə də başqa bir modeldə təkrar istifadə etmək istəyə bilərik):

```

class ResidualBlock(tf.keras.layers.Layer):
    def __init__(self, n_layers, n_neurons, **kwargs):
        super().__init__(**kwargs)
        self.hidden = [tf.keras.layers.Dense(n_neurons, activation="relu",
kernel_initializer="he_normal")
                        for _ in range(n_layers)]
    def call(self, inputs):
        Z = inputs
        for layer in self.hidden:
            Z = layer(Z)

        return inputs + Z

```

Bu qat bir qədər xüsusi, çünki digər qatları ehtiva edir. Bu, Keras tərəfindən şəffaf şəkildə idarə olunur: o, avtomatik olaraq `hidden` atributunun trackable obyektləri (bu halda qatlar) ehtiva etdiyini aşkar edir, buna görə onların dəyişənləri avtomatik olaraq bu qatın dəyişənlər siyahısına əlavə edilir. Bu sinfin qalan hissəsi özünü izah edir. Sonra, modeli təyin etmək üçün subclassing API-dən istifadə edək:

```

class ResidualRegressor(tf.keras.Model):
    def __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = tf.keras.layers.Dense(30, activation="relu",
kernel_initializer="he_normal")
        self.block1 = ResidualBlock(2, 30)
        self.block2 = ResidualBlock(2, 30)
        self.out = tf.keras.layers.Dense(output_dim)
    def call(self, inputs):
        Z = self.hidden1(inputs)
        for _ in range(1 + 3):
            Z = self.block1(Z)
        Z = self.block2(Z)

        return self.out(Z)

```

Biz konstruktorda qatları yaradıırıq və `call()` metodunda onlardan istifadə edirik.

Bu model daha sonra hər hansı digər model kimi istifadə edilə bilər (onu kompilyasiya edin, fit edin, qiymətləndirin və proqnozlar vermək üçün istifadə edin). Əgər modeli `save()` metodundan istifadə edərək saxlamaq və `tf.keras.models.load_model()` funksiyası ilə yükləmək istəyirsinizsə, həm `ResidualBlock` sinfində, həm də `ResidualRegressor` sinfində `get_config()` metodunu tətbiq etməlisiniz (əvvəllər etdiyimiz kimi). Alternativ olaraq, `save_weights()` və `load_weights()` metodlarından istifadə edərək çəkirləri saxlaya və yükləyə bilərsiniz.

`Model` sinfi `Layer` sinfinin alt sinfidir, buna görə modellər qatlar kimi təyin edilə və istifadə edilə bilər. Lakin modelin əlavə funksiyaları var, əlbəttə ki, onun `compile()`, `fit()`, `evaluate()` və `predict()` metodları (və bir neçə variantı), üstəlik `get_layer()` metodu (adı və ya indeks ilə modelin hər hansı qatını qaytara bilər) və `save()` metodu (və `tf.keras.models.load_model()` və `tf.keras.models.clone_model()` dəstəyi).

İPUCU

Əgər modellər qatlardan daha çox funksionallıq təmin edirsə, niyə hər bir qatı model kimi təyin etməmək? Yaxşı, texniki olaraq edə bilərsiniz, lakin adətən modelin daxili komponentlərini (yəni, qatlar və ya təkrar istifadə edilə bilən qat blokları) modelin özündən (yəni, öyrədəcəyiniz obyektədən) fərqləndirmək daha təmizdir. Birincisi `Layer` sinfini, ikincisi isə `Model` sinfini alt sinifə çevirməlidir.

Bununla, siz sequential API, functional API, subclassing API və ya hətta bunların qarışığından istifadə edərək, bir məqalədə tapdığınız demək olar ki, istənilən modeli təbii və qısa şəkildə qura bilərsiniz. "Demək olar

ki" istənilən model? Bəli, hələ baxmaq lazım olan bir neçə şey var: birincisi, modelin daxili hissələrinə əsaslanan losslar və ya metrikaları necə təyin etmək, ikincisi, xüsusi təlim dövrü (custom training loop) qurmaq.

Model Daxili Hissələrinə Əsaslanan Losslar və Metrikalar

Əvvəllər təyin etdiyimiz xüsusi losslar və metrikalar hamısı etiketlərə və proqnozlara (və istəyə bağlı olaraq sample weights) əsaslanırdı. Elə vaxtlar olacaq ki, modelin digər hissələrinə, məsələn, gizli qatlarının çəkiləri və ya aktivasiyalarına əsaslanan losslar təyin etmək istəyəcəksiniz. Bu, regularizasiya məqsədləri üçün və ya modelin bəzi daxili aspektlərini izləmək üçün faydalı ola bilər.

Model daxili hissələrinə əsaslanan xüsusi loss təyin etmək üçün, onu modelin istədiyiniz hissəsinə əsasən hesablayın, sonra nəticəni `add_loss()` metoduna ötürün. Məsələn, gəlin beş gizli qat və bir çıxış qatından ibarət xüsusi regression MLP modeli quraq. Bu xüsusi model, həmçinin yuxarı gizli qatın üstündə köməkçi bir çıxışa sahib olacaq. Bu köməkçi çıxışla əlaqəli loss *reconstruction loss* (bax Fəsil 17) adlanacaq: bu, rekonstruksiya ilə girişlər arasındakı ortalama kvadrat fərkdir. Bu reconstruction loss-u əsas loss-a əlavə etməklə, biz modeli gizli qatlar vasitəsilə mümkün qədər çox məlumatı qorumağa təşviq edəcəyik – hətta regression tapşırığının özü üçün birbaşa faydalı olmayan məlumatları da. Praktikada, bu loss bəzən ümumiləşdirməni (generalization) yaxşılaşdırır (bir regularizasiya loss-dur). Modelin `add_metric()` metodundan istifadə edərək xüsusi metrika əlavə etmək

də mümkündür. Burada xüsusi reconstruction loss və müvafiq metrikanı olan xüsusi model üçün kod verilmişdir:

```
class ReconstructingRegressor(tf.keras.Model):
    def __init__(self, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.hidden = [tf.keras.layers.Dense(30, activation="relu",
kernel_initializer="he_normal")
                        for _ in range(5)]
        self.out = tf.keras.layers.Dense(output_dim)
        self.reconstruction_mean = tf.keras.metrics.Mean(
            name="reconstruction_error")
    def build(self, batch_input_shape):
        n_inputs = batch_input_shape[-1]
        self.reconstruct = tf.keras.layers.Dense(n_inputs)
    def call(self, inputs, training=False):
        Z = inputs
        for layer in self.hidden:
            Z = layer(Z)
        reconstruction = self.reconstruct(Z)
        recon_loss = tf.reduce_mean(tf.square(reconstruction - inputs))
        self.add_loss(0.05 * recon_loss)
        if training:
            result = self.reconstruction_mean(recon_loss)
            self.add_metric(result)

        return self.out(Z)
```

Bu kodu nəzərdən keçirək:

- Konstruktor beş sıx gizli qat və bir sıx çıxış qatı olan DNN yaradır. Biz həmçinin təlim zamanı reconstruction error-u izləmək üçün Mean streaming metric yaradırıq.
- `build()` metodu modelin girişlərini rekonstruksiya etmək üçün istifadə ediləcək əlavə bir sıx qat yaradır. Onun vahidlərinin sayı girişlərin sayına bərabər olmalıdır və bu rəqəm `build()` metodu çağırılmadan əvvəl bilinmir, buna görə burada yaradılmalıdır.
- `call()` metodu girişləri bütün beş gizli qatdan keçirir, sonra nəticəni rekonstruksiya qatına ötürür, bu da rekonstruksiyanı yaradır.
- Sonra `call()` metodu reconstruction loss-u (rekonstruksiya ilə girişlər arasındakı ortalama kvadrat fərq) hesablayır və

`add_loss()` metodundan istifadə edərək onu modelin loss-lar siyahısına əlavə edir. Qeyd edək ki, biz reconstruction loss-u 0.05-ə vuraraq azaldırıq (bu, tənzimləyə biləcəyiniz bir hiperparametrdir). Bu, reconstruction loss-un əsas loss-u üstələməməsini təmin edir.

- Sonra, yalnız təlim zamanı, `call()` metodu reconstruction metric-i yeniləyir və onu modelə əlavə edir ki, göstərilə bilsin. Bu kod nümunəsi əslində `self.add_metric(recon_loss)` çağırmaqla sadələşdirilə bilər: Keras sizin üçün avtomatik olaraq ortalamanı izləyəcək.

Nəhayət, `call()` metodu gizli qatların çıxışını çıxış qatına ötürür və onun çıxışını qaytarır.

Həm ümumi loss, həm də reconstruction loss təlim zamanı aşağı düşəcək:

```
text
```

```
Epoch 1/5
```

```
363/363 [=====] - 1s 820us/step - loss: 0.7640 - reconstruction_error: 1.2728
```

```
Epoch 2/5
```

```
363/363 [=====] - 0s 809us/step - loss: 0.4584 - reconstruction_error: 0.6340
```

```
[...]
```

Əksər hallarda, indiyə qədər müzakirə etdiyimiz hər şey, hətta mürəkkəb arxitekturalar, loss-lar və metrikalarla belə, qurmaq istədiyiniz hər hansı modeli tətbiq etmək üçün kifayət edəcəkdir. Bununla belə, GANs (bax Fəsil 17) kimi bəzi arxitekturalar üçün təlim dövrünü özünüz fərdiləşdirməli olacaqsınız. Buna çatmadan əvvəl, TensorFlow-da qradiyentlərin avtomatik olaraq necə hesablandığına baxmalıyıq.

Autodiff İstifadə edərək Qrادیyentlərin Hesablanması

Autodiff-in (bax Fəsil 10 və Əlavə B) qradientləri avtomatik hesablamaq üçün necə istifadə edildiyini anlamaq üçün sadə bir oyuncaq funksiyasını nəzərdən keçirək:

```
def f(w1, w2):
```

```
    return 3 * w1 ** 2 + 2 * w1 * w2
```

Əgər hesablama (riyaziyyat) bilirsinizsə, bu funksiyanın w_1 -ə görə qismən törəməsinin $6 * w_1 + 2 * w_2$ olduğunu analitik olaraq tapa bilərsiniz. Onun w_2 -yə görə qismən törəməsinin isə $2 * w_1$ olduğunu da tapa bilərsiniz. Məsələn, $(w_1, w_2) = (5, 3)$ nöqtəsində bu qismən törəmələr müvafiq olaraq 36 və 10-a bərabərdir, buna görə bu nöqtədə qradient vektoru $(36, 10)$ -dur. Lakin əgər bu, bir neyron şəbəkəsi olsaydı, funksiya daha mürəkkəb olardı, adətən on minlərlə parametrlə, və qismən törəmələri əl ilə analitik tapmaq demək olar ki, qeyri-mümkün bir iş olardı. Bir həll, müvafiq parametri çox kiçik bir miqdar dəyişdirərək funksiyanın çıxışının nə qədər dəyişdiyini ölçməklə hər bir qismən törəmənin təxminini hesablamaq ola bilər:

```
>>> w1, w2 = 5, 3
>>> eps = 1e-6
>>> (f(w1 + eps, w2) - f(w1, w2)) / eps
36.000003007075065
>>> (f(w1, w2 + eps) - f(w1, w2)) / eps
```

```
10.000000003174137
```

Düzgün görünür! Bu, kifayət qədər yaxşı işləyir və tətbiq etmək asandır, lakin bu, yalnız bir təxmindir və vacib olaraq, $f()$ -i ən azı hər parametr üçün bir dəfə çağırmaq lazımdır (iki dəfə yox, çünki $f(w_1, w_2)$ -ni bir dəfə hesablaya bilərik). $f()$ -i hər parametr üçün ən azı bir dəfə çağırmaq zərurəti bu yanaşmanı böyük neyron şəbəkələr üçün əlçatmaz edir. Buna

görə, əvəzində reverse-mode autodiff istifadə etməliyik. TensorFlow bunu olduqca sadə edir:

```
w1, w2 = tf.Variable(5.), tf.Variable(3.)
with tf.GradientTape() as tape:
    z = f(w1, w2)
```

```
gradients = tape.gradient(z, [w1, w2])
```

Biz əvvəlcə iki dəyişən `w1` və `w2` təyin edirik, sonra bir `tf.GradientTape` konteksti yaradırıq ki, bu da dəyişəni əhatə edən hər bir əməliyyatı avtomatik olaraq qeyd edəcək və nəhayət bu `tape`-dən `z`-nin hər iki dəyişənə `[w1, w2]` görə qradientlərini hesablamağı xahiş edirik. TensorFlow-un hesabladığı qradientlərə baxaq:

```
>>> gradients
[<tf.Tensor: shape=(), dtype=float32, numpy=36.0>,
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=10.0>]
```

Mükəmməl! Nəticə dəqiqdir (dəqiqlik yalnız float xətalrı ilə məhdudlaşır), həm də `gradient()` metodu neçə dəyişən olmasından asılı olmayaraq qeyd edilmiş hesablamalardan yalnız bir dəfə (tərs qaydada) keçir, buna görə inanılmaz dərəcədə səmərəlidir. Bu, sehrə bənzəyir!

İPUCU

Yaddaşı qorumaq üçün, yalnız mütləq lazım olanı `tf.GradientTape()` blokunun daxilinə qoyun. Alternativ olaraq, `tf.GradientTape()` blokunun daxilində `with tape.stop_recording()` bloku yaradaraq qeydi dayandıra bilərsiniz.

Tape `gradient()` metodu çağırıldıqdan dərhal sonra avtomatik olaraq silinir, buna görə `gradient()`-i iki dəfə çağırmağa çalışsanız, bir istisna alacaqsınız:

```
with tf.GradientTape() as tape:
    z = f(w1, w2)
dz_dw1 = tape.gradient(z, w1) # tensor 36.0 qaytarır

dz_dw2 = tape.gradient(z, w2) # RuntimeError yaradır!
```

Əgər `gradient()`-i birdən çox çağırmaq lazımdırsa, tape-i persistent etməli və işiniz bitdikdə resursları boşaltmaq üçün onu silməlisiniz:

```
with tf.GradientTape(persistent=True) as tape:
    z = f(w1, w2)
dz_dw1 = tape.gradient(z, w1) # tensor 36.0 qaytarır
dz_dw2 = tape.gradient(z, w2) # tensor 10.0 qaytarır, indi işləyir!

del tape
```

Defolt olaraq, tape yalnız dəyişənləri əhatə edən əməliyyatları izləyəcək, buna görə `z`-nin dəyişəndən başqa bir şeyə görə qradientini hesablamağa çalışsanız, nəticə `None` olacaq:

```
c1, c2 = tf.constant(5.), tf.constant(3.)
with tf.GradientTape() as tape:
    z = f(c1, c2)

gradients = tape.gradient(z, [c1, c2]) # [None, None] qaytarır
```

Lakin, siz tape-i istədiyiniz tensorları izləməyə məcbur edə bilərsiniz, onları əhatə edən hər bir əməliyyatı qeyd etmək üçün. Daha sonra bu tensorlara görə qradientləri hesablaya bilərsiniz, sanki onlar dəyişənlərmiş kimi:

```
with tf.GradientTape() as tape:
    tape.watch(c1)
    tape.watch(c2)
```

```
z = f(c1, c2)
```

```
gradients = tape.gradient(z, [c1, c2]) # [tensor 36., tensor  
10.] qaytarır
```

Bu, bəzi hallarda faydalı ola bilər, məsələn, girişlər az dəyişdikdə aktivasiyaların çox dəyişdiyini cəzalandıran bir regularizasiya loss-u tətbiq etmək istəyirsinizsə: loss, aktivasiyaların girişlərə görə qradientinə əsaslanacaq. Girişlər dəyişən olmadığı üçün, tape-ə onları izləməyi söyləməlisiniz.

Əksər hallarda, bir gradient tape tək bir dəyərin (adətən loss) bir sıra dəyərlərə (adətən model parametrləri) görə qradientlərini hesablamaq üçün istifadə olunur. Reverse-mode autodiff burada parlayır, çünki bütün qradientləri bir anda əldə etmək üçün yalnız bir forward pass və bir reverse pass etmək lazımdır. Əgər bir vektorun, məsələn, çoxsaylı lossları ehtiva edən bir vektorun qradientlərini hesablamağa çalışsanız, TensorFlow vektorun cəminin qradientlərini hesablayacaq. Beləliklə, əgər fərdi qradientləri (məsələn, hər bir loss-un model parametrlərinə görə qradientlərini) əldə etmələrsinizsə, tape-in `jacobian()` metodunu çağırmalısınız: o, vektordakı hər bir loss üçün bir dəfə reverse-mode autodiff yerinə yetirəcək (defolt olaraq hamısını paralel). Hətta ikinci dərəcəli qismən törəmələri (Hessians, yəni qismən törəmələrin qismən törəmələri) hesablamaq mümkündür, lakin praktikada bu nadir hallarda lazım olur (bir nümunə üçün bu fəslin notebook-unun "Computing Gradients Using Autodiff" bölməsinə baxın).

Bəzi hallarda, neyron şəbəkənin bir hissəsindən qradientlərin geri yayılmasını dayandırmaq istəyə bilərsiniz. Bunu etmək üçün `tf.stop_gradient()` funksiyasından istifadə etməlisiniz. Bu funksiya

forward pass zamanı girişlərini qaytarır (`tf.identity()` kimi), lakin backpropagation zamanı qradientlərin keçməsinə imkan vermir (sabit kimi davranır):

```
def f(w1, w2):  
    return 3 * w1 ** 2 + tf.stop_gradient(2 * w1 * w2)  
with tf.GradientTape() as tape:  
    z = f(w1, w2) # forward pass stop_gradient() tərəfindən təsirlənmir  
  
    gradients = tape.gradient(z, [w1, w2]) # [tensor 30., None]  
    qaytarır
```

Nəhayət, bəzən qradientləri hesablayarkən ədədi problemlərlə qarşılaşa bilərsiniz. Məsələn, kvadrat kök funksiyasının qradientini -50 $x = 10$ nöqtəsində hesablasanız, nəticə sonsuz olacaq. Əslində, o nöqtədəki yamac sonsuz deyil, lakin 32-bit float-ların idarə edə biləcəyindən daha böyükdür:

```
>>> x = tf.Variable(1e-50)  
>>> with tf.GradientTape() as tape:  
...     z = tf.sqrt(x)  
...  
>>> tape.gradient(z, [x])  
  
[<tf.Tensor: shape=(), dtype=float32, numpy=inf>]
```

Bunu həll etmək üçün, kvadrat kökünü hesablayarkən x -ə çox kiçik bir dəyər əlavə etmək çox vaxt yaxşı fikirdir (məsələn, 10^{-6}).

Ekspensial funksiya da tez-tez baş ağrısı mənbəyidir, çünki o, son dərəcə sürətlə böyüyür. Məsələn, `my_softplus()` funksiyasının əvvəllər təyin edildiyi kimi tətbiqi ədədi cəhətdən dayanıqlı deyil. `my_softplus(100.0)` hesablasanız, düzgün nəticə əvəzinə (təxminən 100) sonsuz alacaqsınız. Lakin funksiyanı ədədi cəhətdən dayanıqlı etmək üçün yenidən yazmaq mümkündür: `softplus` funksiyası $\log(1 +$

$\exp(z)$) kimi təyin olunur, bu həmçinin $\log(1 + \exp(-|z|)) + \max(z, 0)$ bərabərdir (riyazi sübut üçün notebook-a baxın) və bu ikinci formanın üstünlüyü ondadır ki, eksponensial termin partlaya bilməz. Beləliklə, `my_softplus()` funksiyasının daha yaxşı tətbiqi:

```
def my_softplus(z):
```

```
    return tf.math.log(1 + tf.exp(-tf.abs(z))) + tf.maximum(0., z)
```

Bəzi nadir hallarda, ədədi cəhətdən dayanıqlı bir funksiya hələ də ədədi cəhətdən qeyri-sabit qradientlərə malik ola bilər. Belə hallarda, TensorFlow-a autodiff istifadə etmək əvəzinə qradientlər üçün hansı tənliyi istifadə edəcəyini söyləməlisiniz. Bunun üçün funksiyanı təyin edərkən `@tf.custom_gradient` dekoratorundan istifadə etməli və həm funksiyanın adi nəticəsini, həm də qradientləri hesablayan bir funksiyanı qaytarmalısınız. Məsələn, `my_softplus()` funksiyasını yeniləyərək ədədi cəhətdən dayanıqlı bir qradient funksiyasını qaytararaq:

```
@tf.custom_gradient
def my_softplus(z):
    def my_softplus_gradients(grads):  # grads = yuxarı qatlardan geri
        yayılmış qradientlər
        return grads * (1 - 1 / (1 + tf.exp(z)))  # softplus-un dayanıqlı
        qradientləri
        result = tf.math.log(1 + tf.exp(-tf.abs(z))) + tf.maximum(0., z)

    return result, my_softplus_gradients
```

Əgər diferensial hesablama bilirsinizsə (bu mövzuda tutorial notebook-a baxın), $\log(1 + \exp(z))$ -nin törəməsinin $\exp(z) / (1 + \exp(z))$ olduğunu tapa bilərsiniz. Lakin bu forma dayanıqlı deyil: z -nin böyük dəyərləri üçün sonsuz bölünən sonsuz hesablanır, bu da NaN qaytarır. Lakin bir az cəbri manipulyasiya ilə onun $1 - 1 / (1 + \exp(z))$ -yə bərabər olduğunu

göstərmək olar, bu da *dayanıqlıdır*. `my_softplus_gradients()` funksiyası qradiyentləri hesablamq üçün bu tənlikdən istifadə edir. Qeyd edək ki, bu funksiya giriş olaraq, `my_softplus()` funksiyasına qədər geri yayılmış qradiyentləri alacaq və zəncir qaydasına görə, biz onları bu funksiyanın qradiyentləri ilə vurmaliyiq.

İndi `my_softplus()` funksiyasının qradiyentlərini hesabladıqda, hətta böyük giriş dəyərləri üçün də düzgün nəticə alırıq.

Təbrik edirik! Siz indi istənilən funksiyanın qradiyentlərini hesablaya bilərsiniz (onları hesabladığınız nöqtədə diferensiallana bilən olmaq şərtilə), hətta lazım olduqda backpropagation-u bloklaya və öz qradiyent funksiyalarınızı yazı bilərsiniz! Bu, yəqin ki, ehtiyacınız olacaqdən daha çox çeviklikdir, hətta öz xüsusi təlim dövrlərinizi qursanız belə. Bunu necə edəcəyinizi növbəti bölmədə görəcəksiniz.

Xüsusi Təlim Dövləri

Bəzi hallarda, `fit()` metodu ehtiyaclarınız üçün kifayət qədər çevik olmaya bilər. Məsələn, Wide & Deep məqaləsində (Fəsil 10-da müzakirə edilmişdir) iki fərqli optimizatordan istifadə edilmişdir: biri wide path üçün, digəri deep path üçün. `fit()` metodu yalnız bir optimizator (modeli kompilyasiya edərkən təyin etdiyimiz) istifadə etdiyi üçün, bu məqaləni tətbiq etmək öz xüsusi dövrünüzü yazmağı tələb edir.

Siz həmçinin, `fit()` metodunun bəzi detalları haqqında əmin deyilsinizsə, özünüzü daha inamlı hiss etmək üçün xüsusi təlim dövrləri yazmağı xoşlaya bilərsiniz. Bəzən hər şeyi açıq etmək daha təhlükəsiz hiss etdirə bilər. Bununla belə, xüsusi təlim dövrü yazmağın kodunuzu

daha uzun, daha çox səhvə meyli və saxlamağı daha çətin edəcəyini unutmayın.

İPUCU

Öyrənmə məqsədilə deyilsinizsə və ya həqiqətən əlavə çevikliyə ehtiyacınız yoxdursa, xüsusi təlim dövrü tətbiq etmək əvəzinə `fit()` metodundan istifadə etməyə üstünlük verməlisiniz, xüsusən bir komandada işləyirsinizsə.

Əvvəlcə, sadə bir model quraq. Onu kompilyasiya etməyə ehtiyac yoxdur, çünki təlim dövrünü əl ilə idarə edəcəyik:

```
l2_reg = tf.keras.regularizers.l2(0.05)
model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(30, activation="relu",
kernel_initializer="he_normal",
kernel_regularizer=l2_reg),
    tf.keras.layers.Dense(1, kernel_regularizer=l2_reg)
])
```

Sonra, təlim setindən təsadüfi bir batch nümunəsi götürəcək kiçik bir funksiya yaradaq (Fəsil 13-də `tf.data` API-ni müzakirə edəcəyik, o, daha yaxşı bir alternativ təklif edir):

```
def random_batch(X, y, batch_size=32):
    idx = np.random.randint(len(X), size=batch_size)

    return X[idx], y[idx]
```

Həmçinin, təlim statusunu, o cümlədən addımların sayını, ümumi addımların sayını, dövrün əvvəlindən ortalama loss-u (bunu hesablamaq üçün Mean metric istifadə edəcəyik) və digər metrikaları göstərən bir funksiya təyin edək:

```
def print_status_bar(step, total, loss, metrics=None):
    metrics = " - ".join([f"{m.name}: {m.result():.4f}" for m in [loss] +
(metrics or [])])
    end = "" if step < total else "\n"

    print(f"\r{step}/{total} - " + metrics, end=end)
```

Bu kod özünü izah edir, əgər Python string formatlaşdırması ilə tanış deyilsinizsə: `{m.result():.4f}` metrikanın nəticəsini onluq nöqtədən sonra dörd rəqəmlə float kimi formatlaşdıracaq və `\r` (carriage return) ilə birlikdə `end=""` istifadəsi status barın hər zaman eyni sətirdə çap olunmasını təmin edir.

Bununla, işə başlayaq! Əvvəlcə, bəzi hiperparametrləri təyin etməli və optimizatoru, loss funksiyasını və metrikaları (bu nümunədə sadəcə MAE) seçməliyik:

```
n_epochs = 5
batch_size = 32
n_steps = len(X_train) // batch_size
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
loss_fn = tf.keras.losses.mean_squared_error
mean_loss = tf.keras.metrics.Mean(name="mean_loss")
```

```
metrics = [tf.keras.metrics.MeanAbsoluteError()]
```

İndi xüsusi dövrü qurmağa hazırıq!

```
for epoch in range(1, n_epochs + 1):
    print("Epoch {}/{}".format(epoch, n_epochs))
    for step in range(1, n_steps + 1):
        X_batch, y_batch = random_batch(X_train_scaled, y_train)
        with tf.GradientTape() as tape:
            y_pred = model(X_batch, training=True)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)
            gradients = tape.gradient(loss, model.trainable_variables)
            optimizer.apply_gradients(zip(gradients,
model.trainable_variables))
        mean_loss(loss)
```

```

for metric in metrics:
    metric(y_batch, y_pred)
print_status_bar(step, n_steps, mean_loss, metrics)
for metric in [mean_loss] + metrics:

    metric.reset_states()

```

Bu kodda çox şey var, buna görə onu nəzərdən keçirək:

- Biz iki iç-içə dövr yaradıırıq: biri dövrlər üçün, digəri dövr daxilindəki batch-lər üçün.
- Sonra təlim setindən təsadüfi bir batch nümunəsi götürürük.
- `tf.GradientTape()` blokunun daxilində, modeli bir funksiya kimi istifadə edərək bir batch üçün proqnoz edirik və loss-u hesablayırıq: o, əsas loss üstəgəl digər losslara (bu modeldə hər qat üçün bir regularizasiya loss var) bərabərdir. `mean_squared_error()` funksiyası hər instans üçün bir loss qaytardığı üçün, batch üzərində ortalamanı `tf.reduce_mean()` ilə hesablayırıq (əgər hər bir instansa fərqli çəkilər tətbiq etmək istəyirsinizsə, bunu burada edərdiniz). Regularizasiya lossları artıq hər biri tək skalyara qədər azaldılıb, buna görə sadəcə onları toplamalıyıq (`tf.add_n()` istifadə edərək, bu, eyni forma və data type-a malik çoxsaylı tensorları toplayır).
- Sonra, tape-dən loss-un hər bir trainable dəyişənə görə qradientlərini hesablamağı xahiş edirik – *bütün* dəyişənlər üçün yox! – və onları optimizatora tətbiq edərək gradient descent addımını yerinə yetiririk.
- Sonra ortalama loss-u və metrikaları (cari dövr üzərində) yeniləyirik və status barı göstəririk.
- Hər bir dövrün sonunda, ortalama loss-un və metrikaların vəziyyətlərini sıfırlayırıq.

Əgər gradient clipping (bax Fəsil 11) tətbiq etmək istəyirsinizsə, optimizatorun `clipnorm` və ya `clipvalue` hiperparametrini təyin edin. Qradyentlərə hər hansı digər transformasiya tətbiq etmək istəyirsinizsə, `apply_gradients()` metodunu çağırmazdən əvvəl bunu edin. Əgər modelinizə çəki məhdudiyyətləri əlavə etmək istəyirsinizsə (məsələn, bir qat yaradarkən `kernel_constraint` və ya `bias_constraint` təyin

etməklə), təlim dövrünü `apply_gradients()`-dən dərhal sonra bu məhdudiyyətləri tətbiq edəcək şəkildə yeniləməlisiniz:

```
for variable in model.variables:  
    if variable.constraint is not None:
```

```
        variable.assign(variable.constraint(variable))
```

XƏBƏRDARLIQ

Təlim dövründə modeli çağırarkən `training=True` təyin etməyi unutmayın, xüsusən modeliniz təlim və test zamanı fərqli davranırsa (məsələn, BatchNormalization və ya Dropout istifadə edirsə). Əgər bu, xüsusi bir modeldirsə, `training` argumentini modelin çağırdığı qatlara ötürdüyünüzə əmin olun.

Gördüyünüz kimi, düzgün etməli olduğunuz bir çox şey var və səhv etmək asandır. Lakin müsbət tərəfdən, tam nəzarət əldə edirsiniz.

İndi modellərinizin və təlim alqoritmlərinizin hər hansı bir hissəsini fərdiləşdirə bildiyinizə görə, TensorFlow-un avtomatik qrafik yaratma xüsusiyyətindən necə istifadə edəcəyinizi görək: bu, xüsusi kodunuzu əhəmiyyətli dərəcədə sürətləndirə bilər və həmçinin onu TensorFlow-un dəstəklədiyi istənilən platformaya daşına bilən edəcək (bax Fəsil 19).

TensorFlow Funksiyaları və Qrafikləri

TensorFlow 1-də, qrafiklər qaçılmaz idi (və onlarla gələn mürəkkəbliklər də) çünki onlar TensorFlow API-sinin mərkəzi hissəsi idilər. TensorFlow 2-dən (2019-cu ildə buraxılmışdır) bəri, qrafiklər hələ də mövcuddur, lakin mərkəzi deyillər və onlardan istifadə etmək daha (daha!) sadədir. Nə

qədər sadə olduğunu göstərmək üçün, girişinin kubunu hesablayan sadə bir funksiya ilə başlayaq:

```
def cube(x):
```

```
    return x ** 3
```

Biz bu funksiyanı açıq-aydın Python dəyəri ilə, məsələn, int və ya float ilə çağıra bilərik, və ya bir tensor ilə çağıra bilərik:

```
>>> cube(2)
```

```
8
```

```
>>> cube(tf.constant(2.0))
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=8.0>
```

İndi, bu Python funksiyanı *TensorFlow funksiyasına* çevirmək üçün `tf.function()` istifadə edək:

```
>>> tf_cube = tf.function(cube)
```

```
>>> tf_cube
```

```
<tensorflow.python.eager.def_function.Function          at  
0x7fbfe0c54d50>
```

Bu TF funksiyası daha sonra orijinal Python funksiyası kimi istifadə edilə bilər və eyni nəticəni qaytaracaq (lakin həmişə tensorlar kimi):

```
>>> tf_cube(2)
```

```
<tf.Tensor: shape=(), dtype=int32, numpy=8>
```

```
>>> tf_cube(tf.constant(2.0))
```

```
<tf.Tensor: shape=(), dtype=float32, numpy=8.0>
```

Alt pərdədə, `tf.function()` `cube()` funksiyası tərəfindən yerinə yetirilən hesablamaları təhlil etdi və ekvivalent bir hesablama qrafiki yaratdı! Gördüyünüz kimi, bu olduqca ağırsız idi (qısa müddətdə bunun necə

işlədiyinə baxacağıq). Alternativ olaraq, `tf.function`-ı dekorator kimi istifadə edə bilərdik; bu, əslində daha çox yayılmışdır:

```
@tf.function
def tf_cube(x):
```

```
    return x ** 3
```

Orijinal Python funksiyası, ehtiyac olduqda, TF funksiyasının `python_function` atributu vasitəsilə hələ də mövcuddur:

```
>>> tf_cube.python_function(2)
```

```
8
```

TensorFlow hesablama qrafikini optimallaşdırır, istifadə olunmayan nodları kəsir, ifadələri sadələşdirir (məsələn, $1 + 2 \cdot 3$ ilə əvəz olunur) və daha çox. Optimallaşdırılmış qrafik hazır olduqda, TF funksiyası qrafikdəki əməliyyatları müvafiq qaydada (və mümkün olduqda paralel) səmərəli şəkildə icra edir. Nəticədə, TF funksiyası adətən orijinal Python funksiyasından çox daha sürətli işləyəcək, xüsusən mürəkkəb hesablamalar yerinə yetirirsə. Əksər hallarda, bundan artıq bilməyə həqiqətən ehtiyacınız olmayacaq: bir Python funksiyasını gücləndirmək istədikdə, onu TF funksiyasına çevirin. Hamısı budur!

Üstəlik, `tf.function()` çağırarkən `jit_compile=True` təyin etsəniz, TensorFlow qrafikiniz üçün xüsusi kernel-lər yaratmaq üçün *accelerated linear algebra* (XLA) istifadə edəcək, çox vaxt birdən çox əməliyyatı birləşdirəcək. Məsələn, əgər TF funksiyanız `tf.reduce_sum(a * b + c)` çağırırsa, XLA olmadan funksiya əvvəlcə `a * b` hesablamalı və nəticəni müvəqqəti dəyişəndə saxlamalı, sonra həmin dəyişənə `c` əlavə etməli və nəhayət nəticədə `tf.reduce_sum()` çağırmalıdır. XLA ilə, bütün hesablama tək bir kernel-ə kompilyasiya edilir, o, heç bir böyük

müvəqqəti dəyişən istifadə etmədən bir anda `tf.reduce_sum(a * b + c)` hesablayacaq. Bu, nəinki daha sürətli olacaq, həm də dramatik dərəcədə az RAM istifadə edəcək.

Xüsusi loss funksiyası, xüsusi metrika, xüsusi qat və ya hər hansı digər xüsusi funksiya yazıb Keras modelində istifadə etdikdə (bu fəsildə etdiyimiz kimi), Keras avtomatik olaraq funksiyanızı TF funksiyasına çevirir – `tf.function()` istifadə etməyə ehtiyac yoxdur. Beləliklə, əksər hallarda, sehr 100% şəffafdır. Və əgər Keras-ın XLA istifadə etməsini istəyirsinizsə, sadəcə `compile()` metodunu çağırarkən `jit_compile=True` təyin etməlisiniz. Asan!

İPUCU

Xüsusi qat və ya xüsusi model yaradarkən `dynamic=True` təyin etməklə Keras-a Python funksiyalarınızı TF funksiyalarına çevirməməyi söyləyə bilərsiniz. Alternativ olaraq, modelin `compile()` metodunu çağırarkən `run_eagerly=True` təyin edə bilərsiniz.

Defolt olaraq, bir TF funksiyası hər bir unikal giriş forma və data type dəsti üçün yeni bir qrafik yaradır və sonrakı çağırışlar üçün onu cache-də saxlayır. Məsələn, `tf_cube(tf.constant(10))` çağırırsanız, int32 tensorlar üçün `[]` formasında bir qrafik yaradılacaq. Sonra `tf_cube(tf.constant(20))` çağırırsanız, eyni qrafik təkrar istifadə olunacaq. Lakin sonra `tf_cube(tf.constant([10, 20]))` çağırırsanız, int32 tensorlar üçün `[2]` formasında yeni bir qrafik yaradılacaq. TF funksiyalarının polimorfizmi (yəni, müxtəlif argument tipləri və formaları) idarə etməsi belədir. Bununla belə, bu, yalnız tensor argumentləri üçün doğrudur: əgər TF funksiyasına ədədi Python dəyərləri ötürsəniz, hər bir

fərqli dəyər üçün yeni bir qrafik yaradılacaq: məsələn, `tf_cube(10)` və `tf_cube(20)` çağırmaq iki qrafik yaradacaq.

XƏBƏRDARLIQ

Əgər TF funksiyasını müxtəlif ədədi Python dəyərləri ilə dəfələrlə çağırırsanız, çoxlu qrafiklər yaradılacaq, proqramınızı yavaşlatacaq və çox RAM istifadə edəcək (onu boşaltmaq üçün TF funksiyasını silməlisiniz). Python dəyərləri, az sayda unikal dəyərə malik olacaq arqumentlər üçün qorunmalıdır, məsələn, qat başına neyronların sayı kimi hiperparametrlər. Bu, TensorFlow-a modelinizin hər bir variantını daha yaxşı optimallaşdırmağa imkan verir.

AutoGraph və izləmə (Tracing)

Bəs TensorFlow qrafikləri necə yaradır? O, Python funksiyasının mənbə kodunu təhlil edərək bütün nəzarət axını ifadələrini, məsələn, for loop, while loop, if ifadələri, həmçinin break, continue və return ifadələrini tutmaqla başlayır. Bu ilk addım *AutoGraph* adlanır. TensorFlow-un mənbə kodunu təhlil etməli olmasının səbəbi, Python-un nəzarət axını ifadələrini tutmaq üçün başqa heç bir yol təqdim etməməsidir: o, + və * kimi operatorları tutmaq üçün `__add__()` və `__mul__()` kimi sehrli metodlar təqdim edir, lakin `__while__()` və ya `__if__()` sehrli metodları yoxdur. Funksiyanın kodunu təhlil etdikdən sonra, AutoGraph həmin funksiyanın bütün nəzarət axını ifadələrinin müvafiq TensorFlow əməliyyatları ilə əvəz edildiyi təkmilləşdirilmiş bir versiyasını çıxarır, məsələn, loop-lar üçün `tf.nn.loop()` və if ifadələri üçün `tf.nn.cond()`. Məsələn, Şəkil 12-4-də, AutoGraph `sum_squares()` Python funksiyasının mənbə kodunu

təhlil edir və `tf.__sum_squares()` funksiyasını yaradır. Bu funksiyada, `for` loop `loop_body()` funksiyasının tərifı (orijinal `for` loop-un gövdəsini ehtiva edir) və ardınca `for_stmt()` funksiyasına çağırışla əvəz olunur. Bu çağırış hesablama qrafikində müvafiq `tf.while_loop()` əməliyyatını quracaq.

Şəkil 12-4. AutoGraph və tracing istifadə edərək TensorFlow-un qrafikləri necə yaratması

Sonra, TensorFlow bu "təkmilləşdirilmiş" funksiyanı çağırır, lakin argument əvəzinə *symbolic tensor* – heç bir faktiki dəyəri olmayan, yalnız adı, data type-i və forması olan bir tensor – ötürür. Məsələn, `sum_squares(tf.constant(10))` çağırırsanız, `tf.__sum_squares()` funksiyası `int32` tipi və `[]` forması olan bir *symbolic tensor* ilə çağırılacaq. Funksiya *graph mode* rejimində işləyəcək, yəni hər bir TensorFlow əməliyyatı özünü və çıxış tensor(lar)ını təmsil etmək üçün qrafikdə bir node əlavə edəcək (adi rejimdən fərqli olaraq, *eager execution* və ya *eager mode* adlanır). Qrafik rejimində, TF əməliyyatları heç bir hesablama yerinə yetirmir. Qrafik rejimi TensorFlow 1-də defolt rejim idi. Şəkil 12-4-də, `tf.__sum_squares()` funksiyasının argumenti kimi *symbolic tensor* ilə çağırıldığını və tracing zamanı son qrafikin yaradıldığını görə bilərsiniz. Nodlar əməliyyatları, oxlar isə tensorları təmsil edir (həm yaradılan funksiya, həm də qrafik sadələşdirilmişdir).

İPUCU

Yaradılan funksiyanın mənbə kodunu görmək üçün `tf.autograph.to_code(sum_squares.python_function)` çağıra

bilərsiniz. Kod gözəl olmaq üçün nəzərdə tutulmayıb, lakin bəzən debug üçün kömək edə bilər.

TF Funksiyası Qaydaları

Əksər hallarda, TensorFlow əməliyyatlarını yerinə yetirən bir Python funksiyasını TF funksiyasına çevirmək sadədir: onu `@tf.function` ilə bəzəyin və ya Keras-ın sizin üçün qayğısına qalmasına icazə verin. Bununla belə, riayət etməli olduğunuz bir neçə qayda var:

- Əgər hər hansı bir xarici kitabxanayı, o cümlədən NumPy və ya hətta standart kitabxanayı çağırırsınız, bu çağırış yalnız tracing zamanı işləyəcək; qrafikin bir hissəsi olmayacaq. Həqiqətən də, TensorFlow qrafiki yalnız TensorFlow konstruksiyalarını (tensorlar, əməliyyatlar, dəyişənlər, dataset-lər və s.) ehtiva edə bilər. Beləliklə, `np.sum()` əvəzinə `tf.reduce_sum()`, daxili `sorted()` funksiyası əvəzinə `tf.sort()` və s. istifadə etdiyinizə əmin olun (əgər həqiqətən kodun yalnız tracing zamanı işləməsini istəmirsinizsə). Bunun bir neçə əlavə təsiri var:
 - Əgər yalnız `np.random.rand()` qaytaran bir TF funksiyası `f(x)` təyin etsəniz, funksiya trace edildikdə təsadüfi bir ədəd yalnız bir dəfə yaradılacaq, buna görə `f(tf.constant(2.))` və `f(tf.constant(3.))` eyni təsadüfi ədədi qaytaracaq, lakin `f(tf.constant([2., 3.]))` fərqli birini qaytaracaq. Əgər `np.random.rand()`-ı `tf.random.uniform([])` ilə əvəz etsəniz, hər çağırışda yeni bir təsadüfi ədəd yaradılacaq, çünki əməliyyat qrafikin bir hissəsi olacaq.
 - Əgər sizin TensorFlow olmayan kodunuzun yan təsirləri varsa (məsələn, bir şeyi log etmək və ya Python saygacını yeniləmək), bu yan təsirlərin TF funksiyasını hər çağırışınızda baş verməsini gözləməməlisiniz, çünki onlar yalnız funksiya trace edildikdə baş verəcək.
 - İxtiyari Python kodunu `tf.py_function()` əməliyyatına bükə bilərsiniz, lakin bu, performansla mane olacaq, çünki TensorFlow bu kod üzərində heç bir qrafik optimallaşdırması edə bilməyəcək. O, həmçinin portativliyi azaldacaq, çünki qrafik yalnız Python-un mövcud olduğu

(və lazımi kitabxanaların quraşdırıldığı) platformalarda işləyəcək.

- Digər Python funksiyalarını və ya TF funksiyalarını çağırma bilərsiniz, lakin onlar eyni qaydalara riayət etməlidir, çünki TensorFlow onların əməliyyatlarını hesablama qrafikində tutacaq. Qeyd edək ki, bu digər funksiyaların `@tf.function` ilə bəzədilməsinə ehtiyac yoxdur.
- Əgər funksiya TensorFlow dəyişəni (və ya hər hansı digər stateful TensorFlow obyekt, məsələn, dataset və ya queue) yaradırsa, bunu ilk çağırışda və yalnız o zaman etməlidir, əks halda bir istisna alacaqsınız. Dəyişənləri TF funksiyasından kənarda yaratmaq adətən daha yaxşıdır (məsələn, xüsusi qatın `build()` metodunda). Dəyişənə yeni dəyər təyin etmək istəyirsinizsə, `=` operatoru əvəzinə onun `assign()` metodunu çağırduğunuzdan əmin olun.
- Python funksiyanızın mənbə kodu TensorFlow üçün əlçatan olmalıdır. Əgər mənbə kodu əlçatan deyilsə (məsələn, funksiyanızı mənbə koduna giriş verməyən Python shell-də təyin etsəniz və ya istehsalata yalnız kompilyasiya edilmiş `*.pyc` Python fayllarını yerləşdirsəniz), qrafik yaratma prosesi uğursuz olacaq və ya məhdud funksionallığa malik olacaq.
- TensorFlow yalnız bir tensor və ya `tf.data.Dataset` (bax Fəsil 13) üzərində təkrarlanan for loop-larını tutacaq. Buna görə, `for i in range(x)` əvəzinə `for i in tf.range(x)` istifadə etdiyinizə əmin olun, əks halda loop qrafikdə tutulmayacaq. Bunun əvəzinə, tracing zamanı işləyəcək. (Əgər for loop qrafiki qurmaq üçün nəzərdə tutulubsa, bu, istədiyiniz şey ola bilər; məsələn, neyron şəbəkədə hər bir qatı yaratmaq üçün.)
- Həmişə olduğu kimi, performans səbəbləri üçün, mümkün olduqda loop-lar əvəzinə vektorlaşdırılmış tətbiqə üstünlük verməlisiniz.

Xülasə etmək vaxtıdır! Bu fəsildə biz TensorFlow-a qısa baxışla başladırıq, sonra TensorFlow-un aşağı səviyyəli API-sinə, o cümlədən tensorlar, əməliyyatlar, dəyişənlər və xüsusi məlumat strukturlarına baxırıq. Daha sonra bu alətlərdən istifadə edərək Keras API-sinin demək olar ki, hər bir komponentini fərdiləşdirdik. Nəhayət, TF funksiyalarının performansı necə artırma biləcəyinə, AutoGraph və tracing istifadə edərək qrafiklərin necə yaradıldığına və TF funksiyaları yazarkən hansı qaydalara riayət

etmək lazım olduğuna baxdıq (əgər qara qutunu bir az daha açmaq və yaradılan qrafikləri araşdırmaq istəyirsinizsə, Əlavə D-də texniki detallar tapa bilərsiniz).

Növbəti fəsildə, TensorFlow ilə məlumatları səmərəli şəkildə yükləmək və preprocess etmək üçün necə edəcəyimizə baxacağıq.

Məşqlər

1. TensorFlow-u qısa bir cümlə ilə necə təsvir edərdiniz? Onun əsas xüsusiyyətləri hansılardır? Digər populyar deep learning kitabxanalarının adını çəkə bilərsinizmi?
2. TensorFlow NumPy-nin birbaşa əvəzləyicisidirmi? İkisi arasındakı əsas fərqlər hansılardır?
3. `tf.range(10)` və `tf.constant(np.arange(10))` ilə eyni nəticəni alırsınızmi?
4. Adi tensorlardan başqa TensorFlow-da mövcud olan altı digər məlumat strukturunun adını çəkə bilərsinizmi?
5. Siz bir funksiya yazaraq və ya `tf.keras.losses.Loss` sinfini alt sinifə çevirərək xüsusi loss funksiyası təyin edə bilərsiniz. Hər bir variantı nə vaxt istifadə edərdiniz?
6. Eynilə, siz bir funksiyada və ya `tf.keras.metrics.Metric` sinfinin alt sinfi kimi xüsusi metrika təyin edə bilərsiniz. Hər bir variantı nə vaxt istifadə edərdiniz?
7. Nə vaxt xüsusi qat, nə vaxt xüsusi model yaratmalısınız?
8. Öz xüsusi təlim dövrünüzü yazmağı tələb edən bəzi istifadə halları hansılardır?
9. Xüsusi Keras komponentləri ixtiyari Python kodu ehtiva edə bilərmi, yoxsa onlar TF funksiyalarına çevrilə bilən olmalıdır?
10. Bir funksiyanın TF funksiyasına çevrilə bilməsi üçün riayət etməli olduğunuz əsas qaydalar hansılardır?
11. Nə vaxt dinamik Keras modeli yaratmaq lazımdır? Bunu necə edirsiniz? Niyə bütün modellərinizi dinamik etməirsiniz?
12. *Layer normalization* (bu tip qatı Fəsil 15-də istifadə edəcəyik) yerinə yetirən xüsusi qat tətbiq edin:
 - `a.build()` metodu iki trainable çəki α və β təyin etməlidir, hər ikisi `input_shape[-1:]` formasında və `tf.float32` data type-a malikdir. α 1-lərlə, β isə 0-larla ilkinləşdirilməlidir.

- b. `call()` metodu hər bir instansın xüsusiyyətlərinin ortalama μ və standart sapma σ -nı hesablamaqlıdır. Bunun üçün `tf.nn.moments(inputs, axes=-1, keepdims=True)` istifadə edə bilərsiniz, bu, bütün instansların ortalama μ və dispersiyasını σ^2 qaytarır (standart sapma almaq üçün dispersiyanın kvadrat kökünü hesablayın). Sonra funksiya $\alpha \otimes (X - \mu) / (\sigma + \epsilon) + \beta$ hesablamaqlı və qaytarmaqlıdır, burada $\otimes$ element-wise vurmanı (*) təmsil edir və ϵ hamarlaşdırma terminidir (sıfıra bölmənin qarşısını almaq üçün kiçik sabit, məsələn, 0.001).
 - c. Xüsusi qatınızın `tf.keras.layers.LayerNormalization` qatı ilə eyni (və ya demək olar ki, eyni) çıxış istehsal etdiyinə əmin olun.
13. Fashion MNIST dataset-i (bax Fəsil 10) üzərində xüsusi təlim dövrü istifadə edərək bir model öyrədin:
- a. Dövrü, iterasiyanı, ortalama təlim loss-u və hər dövr üzərində ortalama dəqiqliyi (hər iterasiyada yenilənir), həmçinin hər dövrün sonunda validasiya loss-u və dəqiqliyini göstərin.
 - b. Yuxarı qatlar və aşağı qatlar üçün fərqli öyrənmə sürəti ilə fərqli bir optimizator istifadə etməyə çalışın.

Bu məşqlərin həlləri bu fəslin notebook-unun sonunda, <https://homl.info/colab3> ünvanında mövcuddur.